

DEVELOPMENT OF A WEB APPLICATION FOR COVID-19 PREDICTION UTILIZING AN OPTIMAL TRANSFER LEARNING MODEL

*Report submitted to the SASTRA Deemed to be University
in partial fulfillment of the requirements
for the award of the degree of*

Bachelor of Technology

Submitted by

RAMALAKSHMI B

(Reg. No.: 124003241, Computer Science Engineering)

SANDHIYA B

(Reg. No.: 124015082, Information Technology)

May 2024



SASTRA
ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION
DEEMED TO BE UNIVERSITY
(U/S 3 of the UGC Act, 1956)



THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

T H A N J A V U R | K U M B A K O N A M | C H E N N A I



SASTRA

ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION

DEEMED TO BE UNIVERSITY

(U/S 3 of the UGC Act, 1956)



THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

T H A N J A V U R | K U M B A K O N A M | C H E N N A I

SCHOOL OF COMPUTING

THANJAVUR – 613 401

Bonafide Certificate

This is to certify that the project report titled “**Development Of a Web Application For Covid-19 Prediction Utilizing Optimal Transfer Learning Model**” submitted in partial fulfillment of the requirements for the award of the degree of **B. Tech. Computer Science and Engineering / B.Tech. Information Technology** to the SASTRA Deemed to be University, is a bona-fide record of the work done by **Ms. Ramalakshmi B(Reg. No. 124003241)**, **Ms. Sandhiya B(Reg. No. 124015082)**, during the final semester of the academic year 202324, in the School of Computing, under my supervision. This report has not formed the basis for the award of any degree, diploma, associateship, fellowship or other similar title to any candidate of any University.

Signature of Project Supervisor:

Name with Affiliation:

Date:

Project Viva voce held on _____

Examiner 1

Examiner 2



SASTRA

ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION

DEEMED TO BE UNIVERSITY

(U/S 3 of the UGC Act, 1956)



THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

T H A N J A V U R | K U M B A K O N A M | C H E N N A I

SCHOOL OF COMPUTING

THANJAVUR – 613 401

Declaration

We declare that the project report titled “**Development Of a Web Application For Covid-19 Prediction Utilizing Optimal Transfer Learning Model**” submitted by us is an original work done by us under the guidance of **Dr. Manivannan P, Asst.professor-III**, School of Computing, SASTRA Deemed to be University during the final semester of the academic year 2023-24, in the School of Computing. The work is original and wherever, We have used materials from other sources, We have given due credit and cited them in the text of the report. This report has not formed the basis for the award of any degree, diploma, associate-ship, fellowship or other similar title to any candidate of any University.

Signature of the candidate(s) :

Name of the candidate(s) : **SANDHIYA B, RAMALAKSHMI B**

Date :

Acknowledgements

We would like to thank our Honorable Chancellor **Prof. R. Sethuraman** for providing us with an opportunity and the necessary infrastructure for carrying out this project as a part of our curriculum.

We would like to thank our Honorable Vice-Chancellor **Dr. S. Vaidhyasubramaniam** and **Dr. S. Swaminathan**, Dean, Planning & Development, for the encouragement and strategic support at every step of our college life.

We extend our sincere thanks to **Dr. R. Chandramouli**, Registrar, SASTRA Deemed to be University for providing the opportunity to pursue this project.

We extend our heartfelt thanks to **Dr. V. S. Shankar Sriram**, Dean, School of Computing, **Dr. R. Muthaiah**, Associate Dean, Research, **Dr. K. Ramkumar**, Associate Dean, Academics, **Dr. D. Manivannan**, Associate Dean, Infrastructure, **Dr. R. Algeswaran**, Associate Dean, Students Welfare.

Our guide **Dr. Manivannan P, Assistant Professor III**, School of Computing was the driving force behind this whole idea from the start. His deep insight in the field and invaluable suggestions helped us in making progress throughout our project work. We also thank the project review panel members for their valuable comments and insights which made this project better.

We would like to extend our gratitude to all the teaching and non-teaching faculties of the School of Computing who have either directly or indirectly helped us in the completion of the project.

We gratefully acknowledge all the contributions and encouragement from my family and friends resulting in the successful completion of this project. We thank you all for providing me an opportunity to showcase my skills through project.

List of Figures

Figure No.	Title	Page No
Fig.1.1	Transfer Learning	4
Fig.1.2	Resnet50 Architecture	4
Fig.1.3	Densenet201 Architecture	5
Fig.1.4	Vgg16 Architecture	5
Fig.1.5	Vgg19 Architecture	6
Fig.1.6	Inceptionv3 Architecture	6
Fig.1.7	Xception Architecture	7
Fig.1.8	MobileNetv2 Architecture	7
Fig.1.9	Confusion Matrix	9
Fig.4.1	Accuracy plot for model1	16
Fig.4.2	Accuracy plot for model2	16
Fig.4.3	Accuracy plot for model3	17
Fig.4.4	Accuracy plot for model4	17
Fig.4.5	Accuracy plot for model5	18
Fig.4.6	Accuracy plot for model6	18
Fig.4.7	Accuracy plot for model7	19
Fig.4.8	Loss plot for model1	19
Fig.4.9	Loss plot for model2	20
Fig.4.10	Loss plot for model3	20
Fig.4.11	Loss plot for model4	21
Fig.4.12	Loss plot for model5	21
Fig.4.13	Loss plot for model6	22
Fig.4.14	Loss plot for model7	22

Fig.4.15	Classification Report for model1	23
Fig.4.16	Classification Report for model2	23
Fig.4.17	Classification Report for model3	23
Fig.4.18	Classification Report for model4	24
Fig.4.19	Classification Report for model5	24
Fig.4.20	Classification Report for model6	24
Fig.4.21	Classification Report for model7	25
Fig.4.22	Confusion matrix for model1	25
Fig.4.23	Confusion matrix for model2	26
Fig.4.24	Confusion matrix for model3	26
Fig.4.25	Confusion matrix for model4	27
Fig.4.26	Confusion matrix for model5	27
Fig.4.27	Confusion matrix for model6	28
Fig.4.28	Confusion matrix for model7	28
Fig 4.29	Image upload page	29
Fig 4.30	Data Attachment page	29
Fig 4.31	Result page for Covid-19 positive	30
Fig 4.32	Result page for Covid-19 negative	30

List of Tables

Table No	Title	Page No
Table 4.1	Performance Analysis for various models	31
Table 7.1	Parameter specifications for various models	35

ABBREVIATIONS

CNN	Convolutional Neural Network
RESNET50	Residual Network
DENSENET201	Densely Connected Convolutional Network
VGG-16,19	Visual Geometry Group
CXR	Chest X-Ray
TP	True Positive
TN	True Negative
FP	False Positive
FN	False Negative

Abstract

The COVID-19 pandemic has devastating health and economic impacts. Molecular testing for virus detection is time-consuming and costly, leading to challenges in early detection. Limited resources and medical equipment exacerbate the difficulties in managing the pandemic. Our project aim is to deployment of optimal Transfer learning models (ResNet50, DenseNet201, InceptionV3, VGG-16, VGG-19, Xception, and MobileNetV2) for early detection of COVID-19 using chest X-ray images.

The model, trained on a Kaggle dataset with high accuracy, shows promise in aiding early COVID-19 detection, potentially enabling quicker intervention and treatment. During our project, we thoroughly investigated the capabilities of seven different transfer learning models, each presenting a distinct method for addressing our machine learning obstacles.

Our comprehensive assessment procedure focused on identifying the most optimal model based on factors such as accuracy, efficiency, and suitability for our specific task. This model has been successfully deployed and integrated into our project, where it continues to demonstrate its efficacy in addressing our machine learning obstacles.

Keywords: Transfer learning models, early detection, accuracy, efficacy, covid-19 detection.

Specific Contribution:

- ☐ processing using ROI (Region Of Intrest).
- ☐ Model (fine-tuned with hyperparameters).
- ☐ Defining model, testing data and choosing best model.
- ☐ Creating front-end application and using flask to integrate the model

Specific Learning:

- ☐ Utilization of transfer learning models to access various architecture.
- ☐ Xception outperformed all models in COVID-19 detection.
- ☐ Deployment of ml model as user-friendly covid-19 prediction.

Table of Contents

Title	Page No.
Bona-fide Certificate	ii
Declaration	iii
Acknowledgements	iv
List of Figures	v
List of Tables	vi
Abbreviations	vii
Abstract	viii
1. Introduction	
1.1. Problem Statement	1
1.2. Literature Survey	2
1.3. Drawbacks	3
1.4. Transfer Learning Models	4
1.5. Quantitative Analysis	8
1.6. Motivation	9
2. Objectives	10
3. Experimental Work / Methodology	
3.1. Dataset Description	11
3.2. Data Preprocessing	11
3.3. Proposed Model	13
3.4. Testing And Prediction	13
3.5. Web application for user friendly	14
4. Results and Discussion	
4.1. Effect of Epochs	16
4.2. Effect of loss	19
4.3. Model Evaluation	23
4.4. Confusion Matrix	25
4.5. Web Application	29
5. Conclusions and Further Work	32
6. References	34
7. Appendix	
7.1 Sample Source code	35

CHAPTER-1

INTRODUCTION

1.1 PROBLEM STATEMENT:

The global spread of COVID-19 demands rapid and reliable diagnostic methods. Current gold-standard tests, like RT-PCR, have limitations. They can be slow, costly, and require specialized equipment, hindering their use in resource-constrained settings.

Early detection is crucial for containing the virus. Existing methods using chest X-rays (CXRs) often rely on subjective human interpretation, leading to potential inaccuracies. Additionally, some Deep Learning models for CXR analysis suffer from high false-positive rates, causing unnecessary anxiety and resource strain.

This project proposes a novel solution to address these challenges. We aim to develop a highly accurate diagnostic tool using a single, optimal pre-trained Convolutional Neural Network (CNN) architecture for analyzing CXR images. This approach focuses on minimizing false-positive results while achieving high prediction accuracy.

The proposed CNN model, integrated with smart devices within the Internet of Medical Things (IoMT) framework, could revolutionize COVID-19 screening. This would enable remote diagnoses, minimizing patient exposure and alleviating pressure on healthcare facilities. Ultimately, this project aspires to contribute significantly to the fight against COVID-19 by providing a faster, more accessible, and reliable diagnostic tool.

1.2 LITERATURE SURVEY:

Transfer Learning for Chest X-rays (CXRs): This approach leverages pre-trained Convolutional Neural Networks (CNNs) fine-tuned for COVID-19 identification in CXR images. Studies by Rahimzadeh et al. [7] and Apostolopoulos et al. [8] achieved promising results with transfer learning frameworks. Notably, Apostolopoulos et al. found VGG-19 and MobileNetV2 to be particularly effective, achieving a 98.75% accuracy rate for a two-class (COVID-19 and normal) classification.

Loey et al. [9] investigated combining transfer learning with data augmentation techniques and Generative Adversarial Networks (GANs) for CT scan analysis. Their findings suggest that ResNet50 outperformed other models, achieving an accuracy of 81.38%. Similarly, Ko et al. [14] developed a framework called FCONet, where ResNet50 again demonstrated superior performance with a 98.67% accuracy for classifying CT scans.

Several studies have compared the performance of various DL models for COVID-19 detection. Taresh et al. [10] identified VGG-16 as a strong performer for CXR analysis, achieving a 98.72% accuracy rate. Fan et al. [11] reported promising results with MobileNetV2 and Xception models, achieving accuracies of 95% and 96% respectively on CXR images. Beyond CXR analysis, Horry et al. [12] observed VGG-19 outperforming other models on ultrasound images with an F1-score of 0.99. Civit-Masot et al. [13] achieved an AUC value greater than 90% for classifying radiographic images using deep learning techniques.

While most studies focus on binary or multi-class classification (COVID-19 vs. normal/pneumonia), Islam et al. [20] proposed a combined CNN-LSTM model for CXR images that achieved high sensitivity, specificity, and F1-score. This suggests the potential for DL models to progress beyond basic classification tasks.

1.3 DRAWBACKS:

Many studies [7, 8, 11, 15] rely on datasets with a relatively small number of samples. This can lead to overfitting, where the model performs well on the training data but fails to generalize to unseen data.

In some datasets, COVID-19 cases are outnumbered by normal or other types of pneumonia [10]. This class imbalance can make it difficult for models to accurately detect COVID-19, as they may be biased towards the more frequent classes.

Studies by Ko et al. [14] and Jain et al. [18] raise concerns about the generalizability of some models. If the training data is not sufficiently diverse, the model may not perform well on real-world data with different characteristics.

False positives, where the model incorrectly identifies a non-COVID-19 case as positive, can lead to unnecessary anxiety and resource allocation. Some studies [7] have reported high false positive rates, highlighting the need for models with better accuracy and discrimination.

Narayan et al. [16] emphasize the potential of data augmentation techniques like GANs to address limitations caused by small datasets. These techniques can artificially generate additional training data, improving model performance.

Studies like Pham et al. [19] were unable to differentiate between mild, moderate, or severe COVID-19 due to restricted data labeling. More comprehensive datasets with detailed annotations are needed to enable sub-classification of COVID-19 severity.

Wang et al. [17] highlight the importance of expert radiologist validation alongside DL models. While DL models offer significant potential, they should not be solely relied upon for diagnosis.

1.4 TRANSFER LEARNING MODELS:

Transfer learning has been widely adopted, utilizing pre-trained convolutional neural networks(CNNs) like Resnet50,Densenet201,VGG-16, VGG-19,Xception,Inceptionv3 and MobileNetV2 for image classification. These models achieve high accuracy in some cases, exceeding 98% for binary classification (COVID-19 vs. normal).

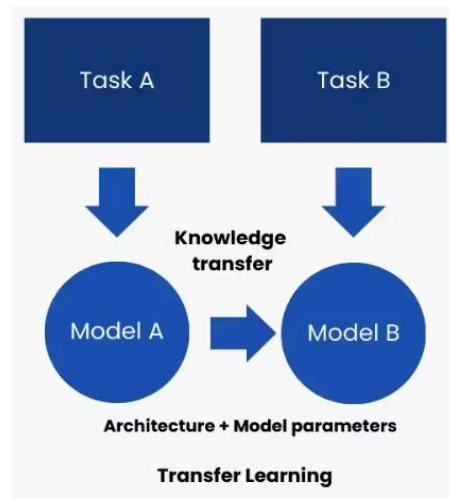


Fig-1.1 Transfer Learning

1.4.1 RESNET50:

ResNet50 is a deep convolutional neural network architecture consisting of 50 layers, designed to solve the vanishing gradient problem in very deep networks. It introduces skip connections, or shortcuts, which allow gradients to flow directly through the network during training, which facilitates the training of deeper networks and improves performance in image recognition tasks.

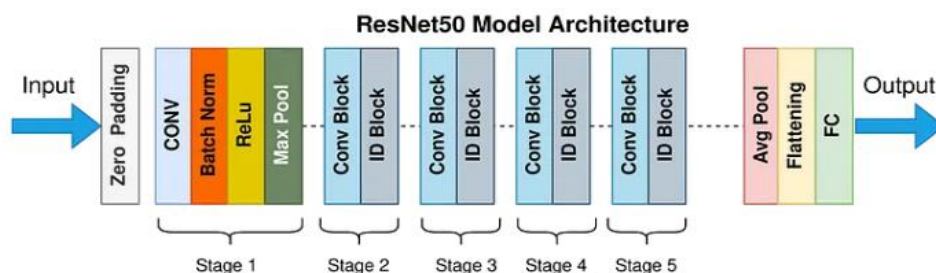


Fig-1.2 Resnet50 Architecture

1.4.2 DENSENET201:

DenseNet201 is a convolutional neural network architecture consisting of densely connected blocks. Each layer is connected to every other layer in a feed-forward fashion, promoting feature reuse and enhancing gradient flow during training. This results in improved parameter efficiency, higher accuracy, and better feature propagation in tasks like image classification and object detection.

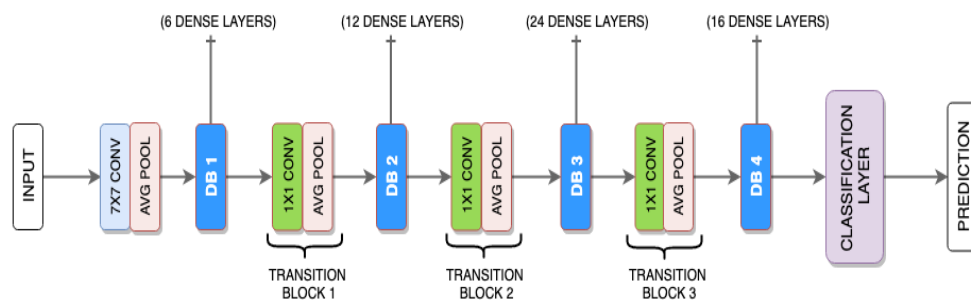


Fig-1.3 Densenet201 Architecture

1.4.3 VGG-16:

VGG-16 is a convolutional neural network architecture with 16 layers, characterized by its simple and uniform design. It stacks multiple 3x3 convolutional layers followed by max-pooling layers to learn hierarchical features and the model achieves competitive performance in image classification tasks due to its effective feature extraction capabilities.

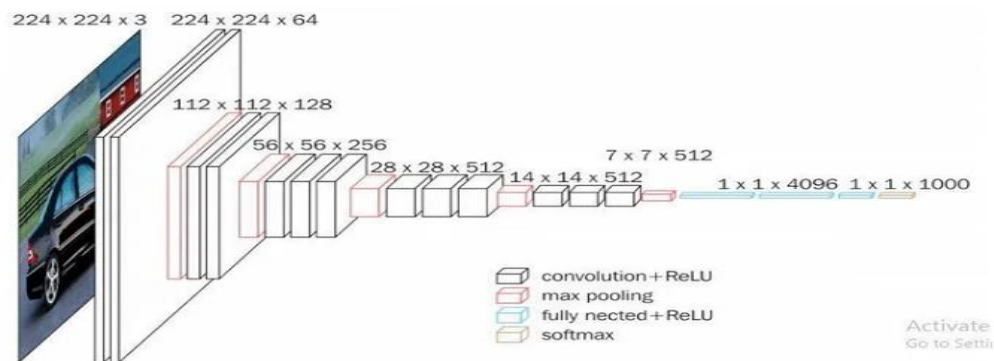


Fig-1.4 vgg16 Architecture

1.4.4 VGG-19:

VGG-19 is a convolutional neural network architecture with 19 layers, an extended version of VGG-16. It maintains the same design principles, stacking multiple 3x3 convolutional layers followed by max-pooling layers. With a deeper architecture, VGG-19 can capture more complex features, making it suitable for tasks like image classification and object recognition with improved performance.

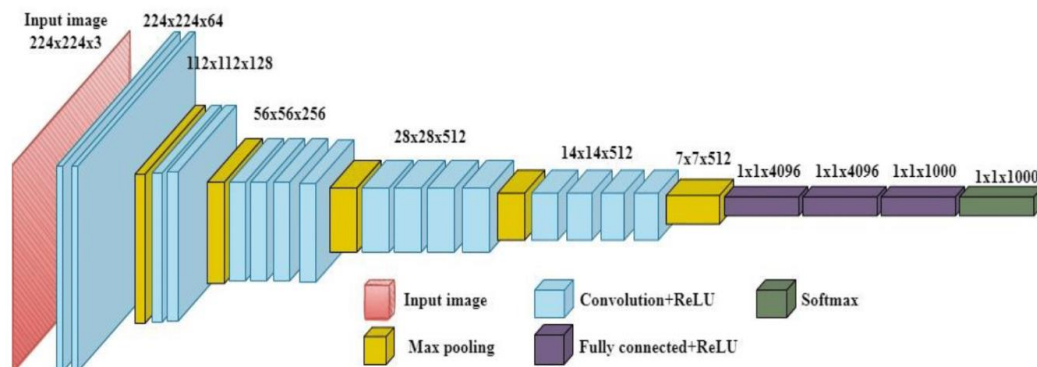


Fig-1.5 vgg19 Architecture

1.4.5 INCEPTIONV3:

InceptionV3 is a convolutional neural network architecture featuring multiple parallel convolutional layers of different sizes. It utilizes 1x1, 3x3, and 5x5 convolutions, along with max-pooling and inception modules, to capture diverse features at various scales efficiently. This design enhances feature extraction and enables robust performance in tasks like image classification and object detection.

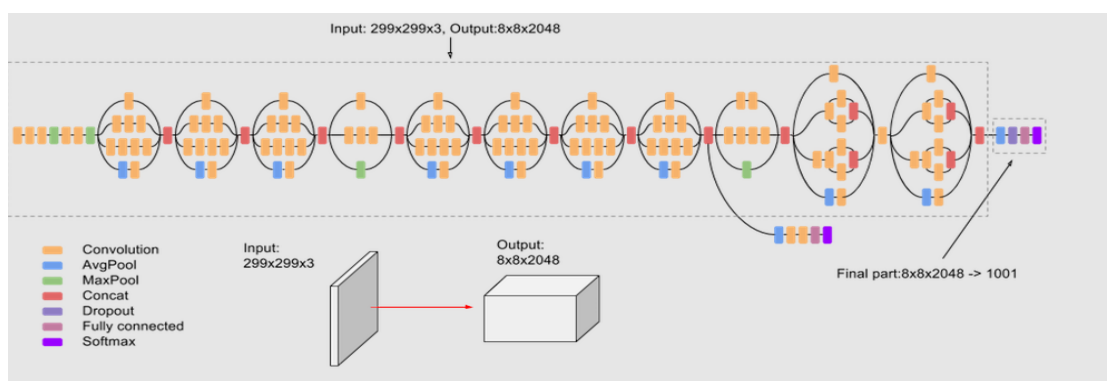
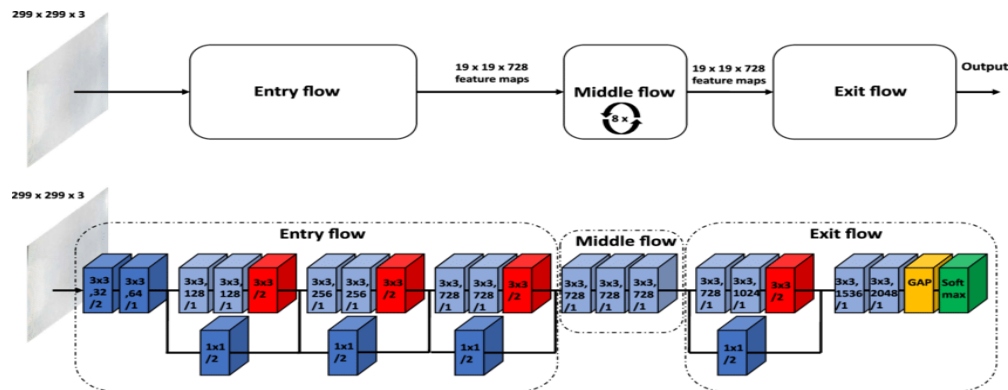


Fig-1.6 InceptionV3 Architecture

1.4.6 XCEPTION:

Xception, a CNN-based transfer learning model, utilizes depth-wise separable convolutions, reducing parameters for faster training. It consists of entry, middle, and exit flows. Data pass through entry and middle flows eight times before the exit flow. Soft Max and binary cross-entropy loss yield binary classification results after the fully connected layer.



1.5 QUANTITATIVE ANALYSIS:

In terms of true positives (TP), true negatives (TN), false positives (FP) and false negatives (FN), the formula of the accuracy can be written as,

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{FN} + \text{TN}}$$

Precision:

Precision is the ratio of true positive (TP) predictions to the total number of positive (TP+FP), predictions made by the model.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Recall:

Recall is the ratio of true positive (TP) predictions to the total number of actual positive (TP + FN) cases in the data.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

F1-Score:

F1-score is the harmonic mean of precision and recall, and is often used as a summary metric that balances the two. F1-score ranges from 0 to 1, with a higher score indicating better performance.

$$\text{F1-Score} = \frac{2 * (\text{RECALL} * \text{PRECISION})}{\text{RECALL} + \text{PRECISION}}$$

Confusion Matrix:

The confusion matrix is a matrix used to determine the performance of the classification models for a given set of data. It is a table with two rows and two columns that represents the number of true positives, false negatives, false positives, and true negatives.

		Prediction	
		1	0
Actual	1	True Positive (TP)	False Negative (FN)
	0	False Positive (FP)	True Negative (TN)

Fig 1.9 Confusion Matrix

1.6 MOTIVATION:

This project investigates the identification of a single, optimal pre-trained Convolutional Neural Network (CNN) architecture for analyzing chest X-rays (CXRs). By focusing on a single model, the development process can be streamlined, while prioritizing high prediction accuracy for COVID-19 detection. Additionally, minimizing false-positive results is crucial to ensure more reliable diagnoses and reduce unnecessary procedures for patients.

A streamlined, optimal model can potentially have lower computational requirements. This opens doors for integration with smart devices within the Internet of Medical Things (IoMT) framework. Such a solution could enable faster remote diagnoses, particularly benefiting areas with limited access to healthcare facilities.

Improved screening efficiency and accuracy through an optimized DL model can lead to a significant reduction in the burden on healthcare facilities and personnel. This would free up valuable resources to focus on patient care and treatment, ultimately improving overall healthcare outcomes.

CHAPTER-2

OBJECTIVE

This project aims to develop a mobile-friendly transfer learning model for rapid and accurate COVID-19 diagnosis using chest X-rays. This model is designed for deployment in resource-limited settings, particularly mobile clinics, to improve public health surveillance in remote areas.

The overarching objective is to create a deep learning model that achieves a high degree of accuracy in COVID-19 case prediction. To achieve this, we will employ a multi-pronged approach:

Curated Dataset and Pre-processing: In the first phase, we will focus on collecting and pre-processing a chest X-ray dataset specifically tailored for COVID-19 classification.

Pre-trained Model Exploration and Identification: Next, we will explore and identify pre-trained deep learning models that have demonstrated success in image classification tasks.

Model Training and Fine-tuning: The chosen pre-trained model will be fine-tuned on our curated dataset to achieve an accuracy of at least 85% sensitivity in detecting COVID-19 cases.

Model Evaluation and Refinement: The final phase will involve evaluating the model's performance using metrics like accuracy, sensitivity, and specificity. Based on these results, we will further refine the model to optimize its effectiveness for web app creation.

By achieving these objectives, this project will culminate in a mobile-friendly transfer learning model that can be readily used in resource-limited settings to facilitate rapid COVID-19 diagnosis and enhance public health surveillance in remote areas.

CHAPTER-3

METHADODOLOGY

3.1 DATASET DESCRIPTION:

The dataset used for this experiment consisting of total of 2000 chest X-ray images. Additionally, each folder includes mask images, making a total of 2000 images across both categories. The model classified the image into 2 classes i) COVID-19 Positive and ii) COVID-19 Negative. Here the dataset is divided into 2 sets. The training set consists of 80% of total data and validation set consists of 20% of the remaining data.

3.2 DATA PREPROCESSING:

3.2.1. Image Loading and Grayscale Conversion:

The initial stage involves reading the chest X-ray images from their designated storage location. Popular image processing libraries like OpenCV can be employed for efficient image loading. Following the loading process, the loaded images are converted into grayscale format. This conversion discards color information, which might be irrelevant for the task of COVID-19 classification based solely on the X-ray's structural details. Grayscale conversion also reduces the computational complexity required for subsequent processing stages.

3.2.2 Noise Reduction using Gaussian Blur:

Natural images often contain inherent noise, which can introduce artifacts and potentially hinder the model's ability to discern crucial features. To mitigate this issue, a Gaussian blur filter is applied to the grayscale images. This filter operates by replacing the value of each pixel with a weighted average of its neighboring pixels. The assigned weights follow a Gaussian distribution, with the central pixel receiving the highest weight and weights decreasing progressively as we move further away. This approach effectively smoothes out the image, suppressing unwanted noise while preserving essential structural details. The parameters of the Gaussian blur filter, such as the kernel size and standard deviation, can be fine-tuned based on the specific characteristics of the dataset and desired level of noise reduction.

3.23. Edge Detection with Canny Edge Filter:

Following noise reduction, the preprocessed images are subjected to edge detection. This step aims to identify and highlight prominent edges within the image, often representing boundaries of anatomical structures in chest X-rays. The Canny edge filter, a widely used technique, is frequently employed for this purpose. This filter incorporates several steps: firstly, it applies a smoothing filter similar to a Gaussian blur to reduce noise. Secondly, it calculates the image gradient magnitude and direction at each pixel. Finally, it employs a non-maximum suppression technique to eliminate spurious edges and hysteresis thresholding to retain only those edges exceeding a certain level of significance. The Canny edge detector effectively locates and delineates crucial edges in the image, offering valuable information for subsequent image analysis tasks.

3.2.4. Contour Detection:

Once the edges are identified, the next step involves detecting contours within the image. Contours represent a sequence of connected points that delineate the boundaries of objects in an image. In the context of chest X-ray analysis, contours might encompass the outlines of the lungs, ribs, and potentially other anatomical structures. Contour detection algorithms typically operate by following the identified edges and tracing connected sequences of pixels that satisfy certain criteria. The extracted contours provide a more detailed representation of the image's prominent features compared to the raw edge map.

3.2.5. Visualization and Region of Interest (ROI) Selection:

For visualization and potential manual inspection, the detected contours are superimposed onto a blank image, creating a visually interpretable representation of the contours within the original image. This visual representation allows for verification of the effectiveness of the contour detection algorithm and enables researchers to assess whether the identified contours capture the desired objects of interest.

3.3 PROPOSED MODEL:

Here, the development of a Conv-neural network for COVID-19 detection using TL. Using seven pre-trained models (RESNET50, DENSENET201, VGG16, VGG19, INCEPTIONV3, XCEPTION, MOBILENETV2) with its convolutional layers frozen, acts as a feature extractor. Three models (VGG-16, XCEPTION, INCEPTIONV3) underwent hyperparameter tuning to enhance their accuracy. This leverages the model's knowledge gained from the massive ImageNet dataset for general image recognition, providing a strong foundation for classifying COVID-19 chest X-rays.

A custom classification head is added on top. It flattens the extracted features and feeds them into a dense layer with 256 neurons and ReLU activation for non-linearity. Finally, binary classifier with sigmoid output layer which tackles the binary classification task of identifying COVID-19 presence.

Employing a learning rate of 0.001, the Adam optimizer directs the trajectory of the training process. The model is compiled with a binary cross-entropy loss function and accuracy metric. Early stopping prevents overfitting by monitoring validation loss and halting training if it stagnates for a set number of epochs. The model is then trained for 10 epochs to achieve optimal differentiation between COVID-positive and negative X-ray images.

3.4 TESTING AND PREDICTION:

A fundamental aspect of machine learning model development is evaluating the model's generalizability to unseen data. To achieve this objective, a separate test set, meticulously isolated from the training data, is employed. This rigorous approach ensures an unbiased assessment of the model's performance on COVID-19 cases it has not encountered during training. The test set comprises a collection of chest X-ray images meticulously categorized as either positive or negative for COVID-19. These images are kept within designated folders for efficient access and organization.

The core functionality lies in the custom procedure which is meticulously designed to process uploaded images. This function resizes, preprocesses, and expands the image for compatibility with the model. It then predicts the probability of the image

belonging to the COVID-19 positive class. A threshold converts this probability into a binary classification ("COVID-19 positive" or "COVID-19 negative").

This evaluation process on unseen data offers invaluable insights into the model's generalizability. The model's ability to accurately classify unseen COVID-19 cases is a crucial indicator of its potential effectiveness in real-world clinical applications.

3.5WEB APPLICATION FOR USER-FRIENDLY INTERFACE:

A Flask web application facilitates user interaction with the trained model. This application provides a user-friendly interface for COVID-19 detection tasks. Pre-defined functions seamlessly load the serialized model for real-world image classification.

The application utilizes several pre-trained deep learning models specifically designed for image classification tasks. The user defined function is employed to load each model from an HDF5 file, a common format for storing deep learning models. This function ensures that the pre-trained models are readily available for image prediction within the application.

The image is read using OpenCV, a popular library for computer vision tasks. The function then resizes the image to a specific dimension required by the pre-trained models. Additionally, the image is converted into a format suitable for neural network processing and normalized by dividing pixel values by 255. This normalization step ensures all pixel intensities fall within a specific range, promoting better model performance.

The pre-processed image is subsequently fed into each of the loaded models (model1 to model7). Each model produces a probability score for the "COVID-19 Positive" class.

A pre-defined threshold (e.g., 0.5) is used to convert the predicted probability scores into binary class labels ("COVID-19 Positive" or "Normal"). The application currently utilizes an ensemble approach, where predictions from all seven models are combined into a dictionary named predictions. This dictionary holds the classification results for each model, formatted as strings indicating the predicted class and corresponding probability.

The application manages user interactions through defined routes. Users upload images, and the application validates the format. The predicted class label is displayed, bridging the gap between the model and user experience.

CHAPTER-4

RESULTS

4.1 EFFECT OF EPOCHS:

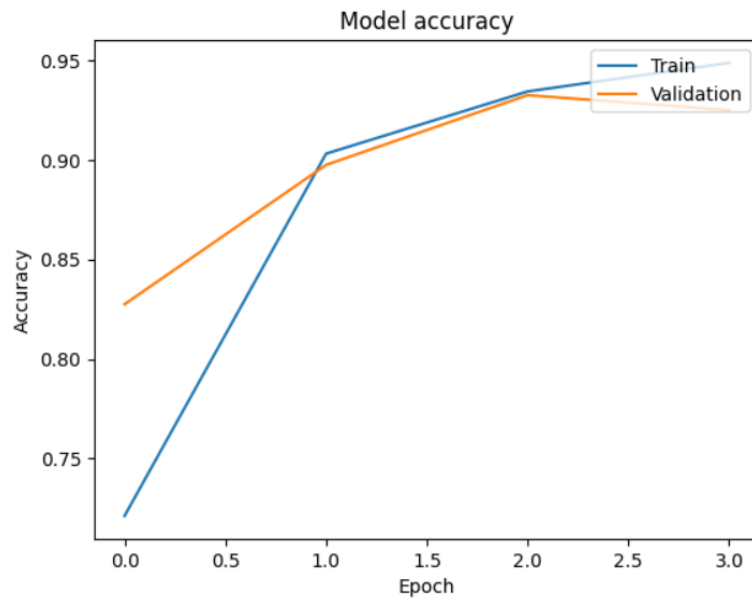


Fig 4.1 Accuracy plot for model1

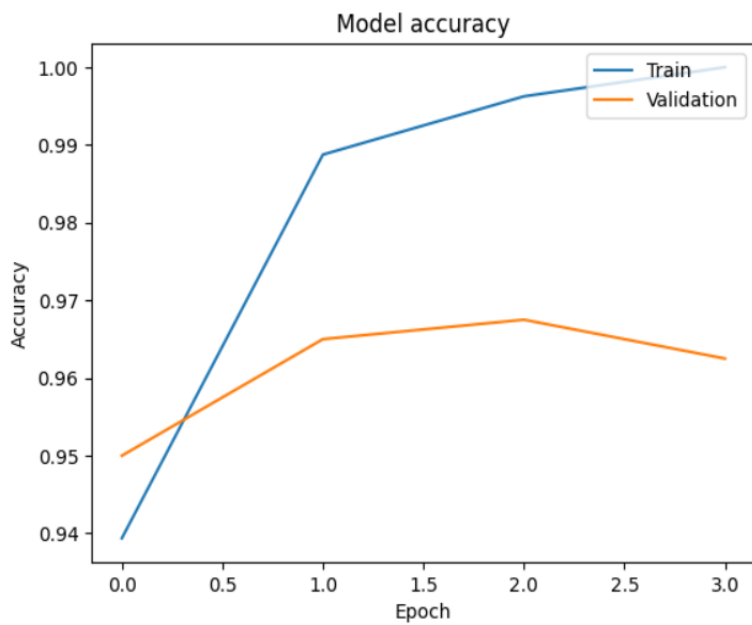


Fig 4.2 Accuracy plot for model2

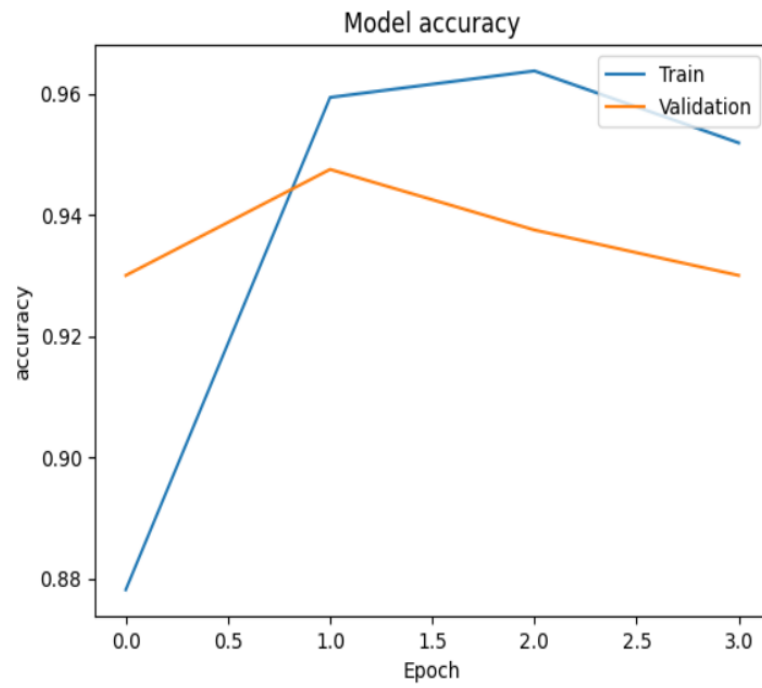


Fig 4.3 Accuracy plot for model3

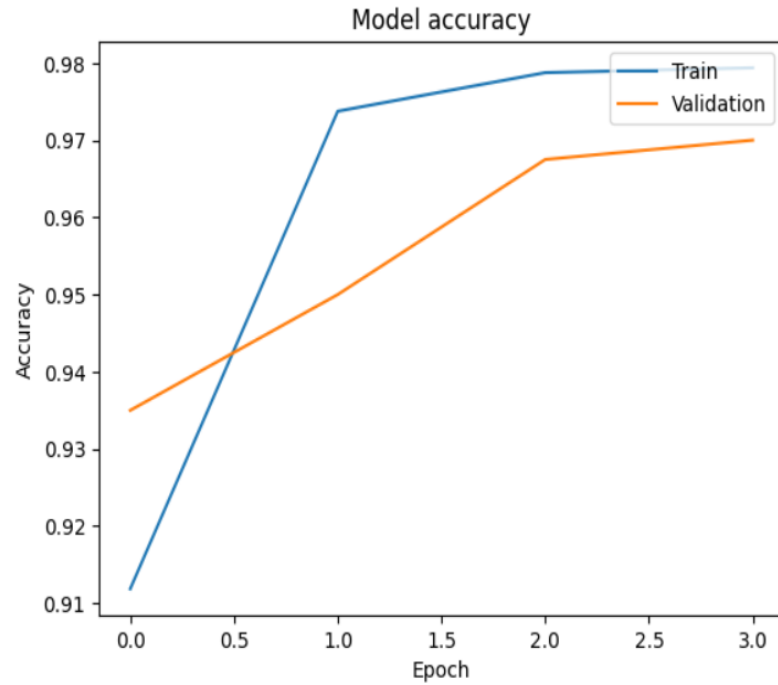


Fig 4.4 Accuracy plot for model4

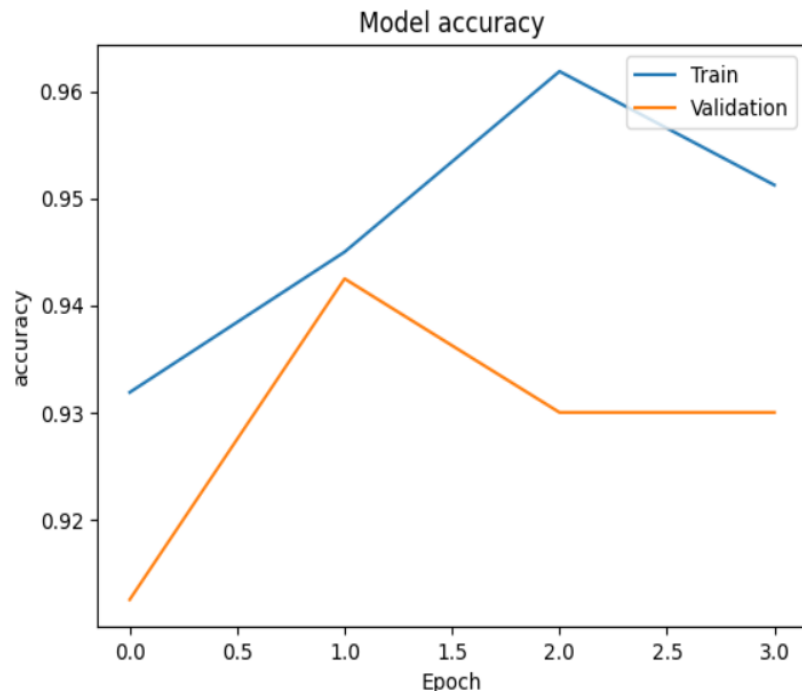


Fig 4.5 Accuracy plot for model5

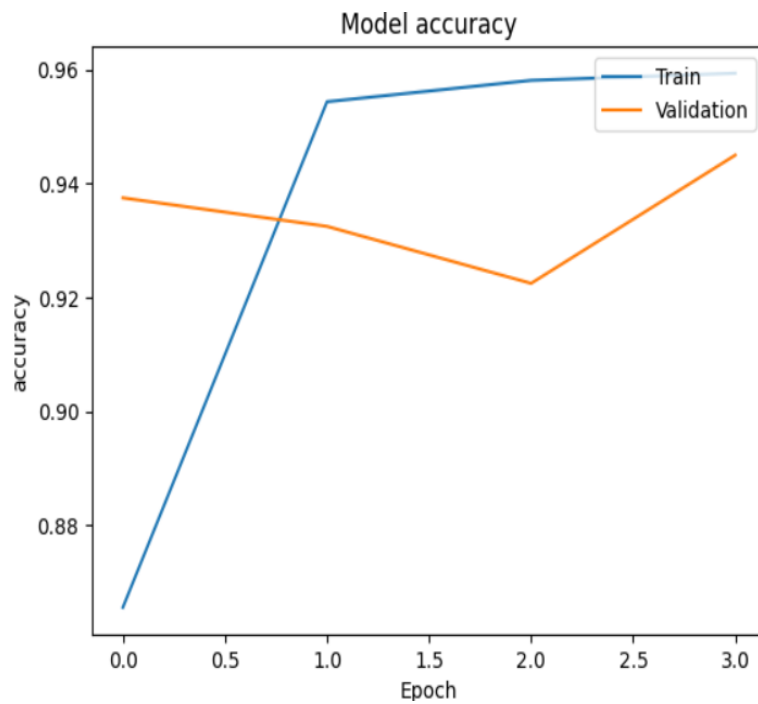


Fig 4.6 Accuracy plot for model6

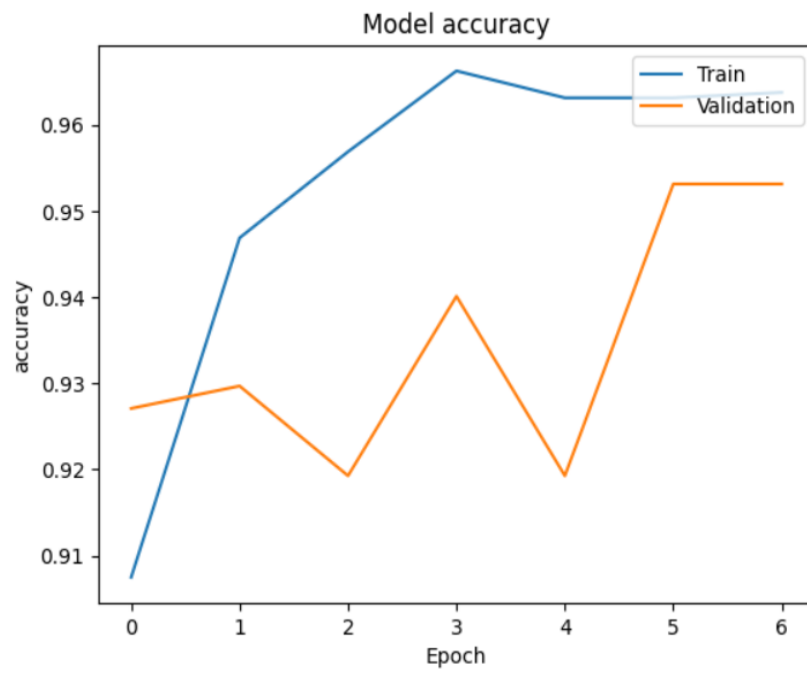


Fig 4.7 Accuracy plot for model7

4.2 EFFECT OF LOSS:

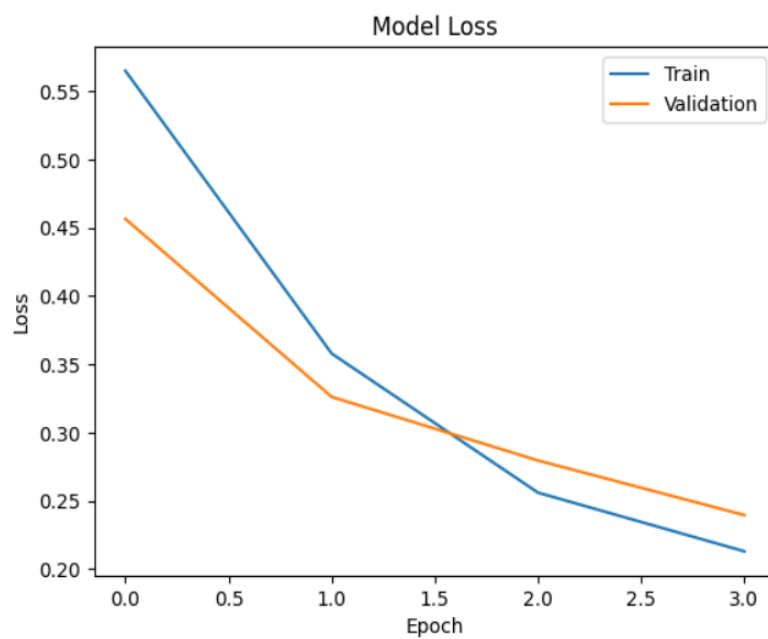


Fig 4.8 Loss plot for model1

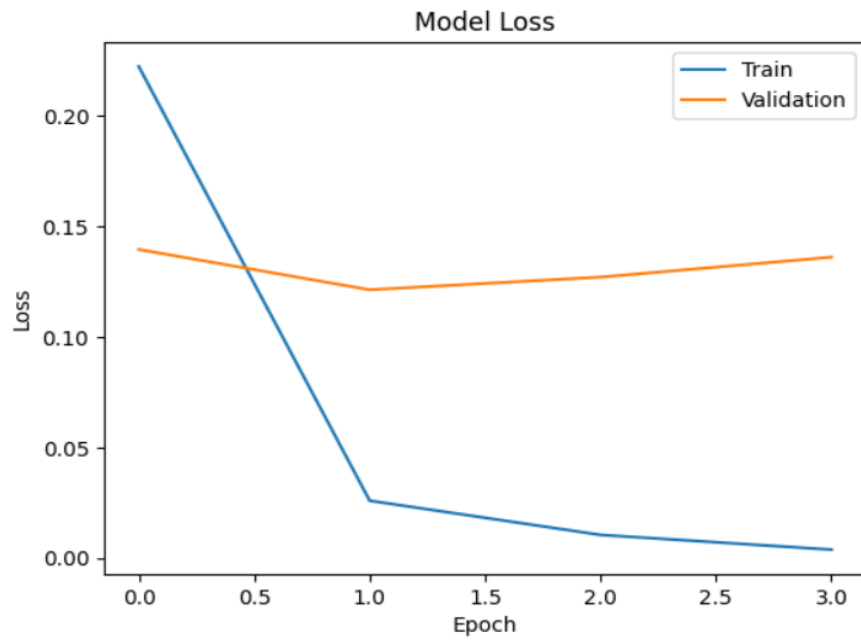


Fig 4.9 Loss plot for model2

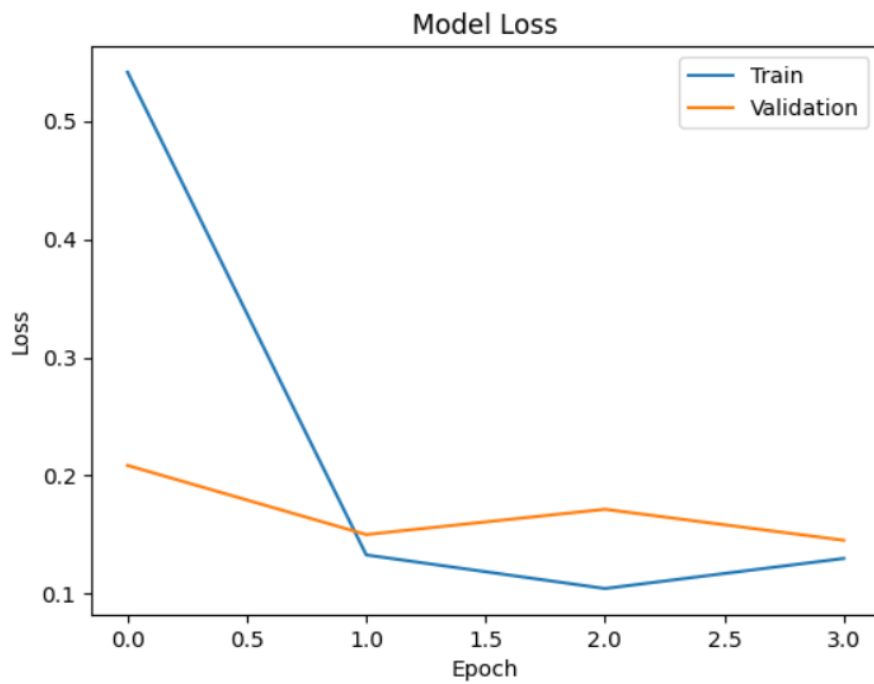


Fig 4.10 Loss plot for model3

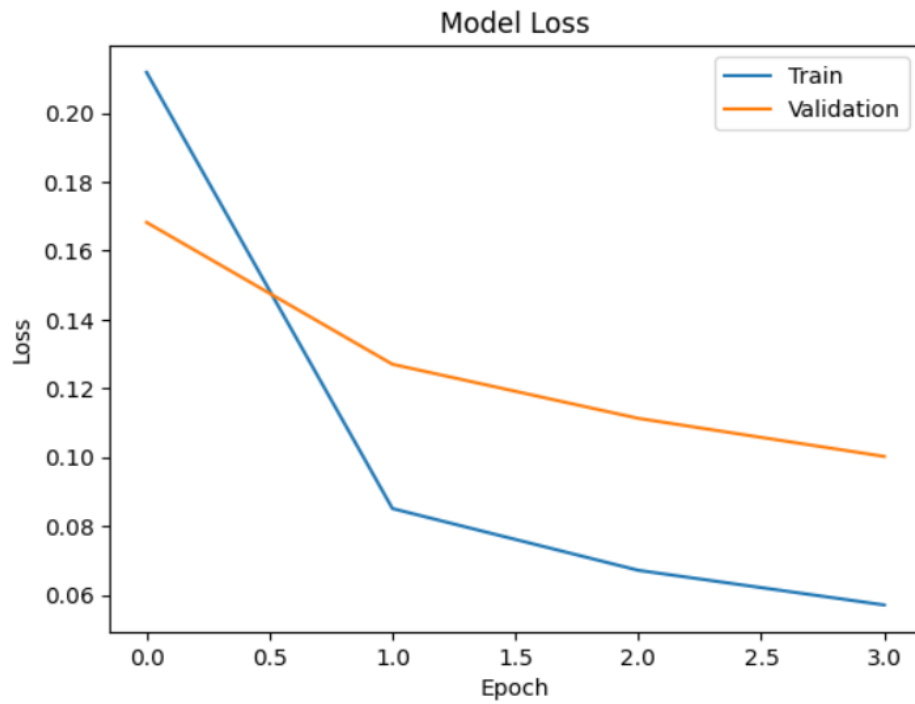


Fig 4.11 Loss plot for model4

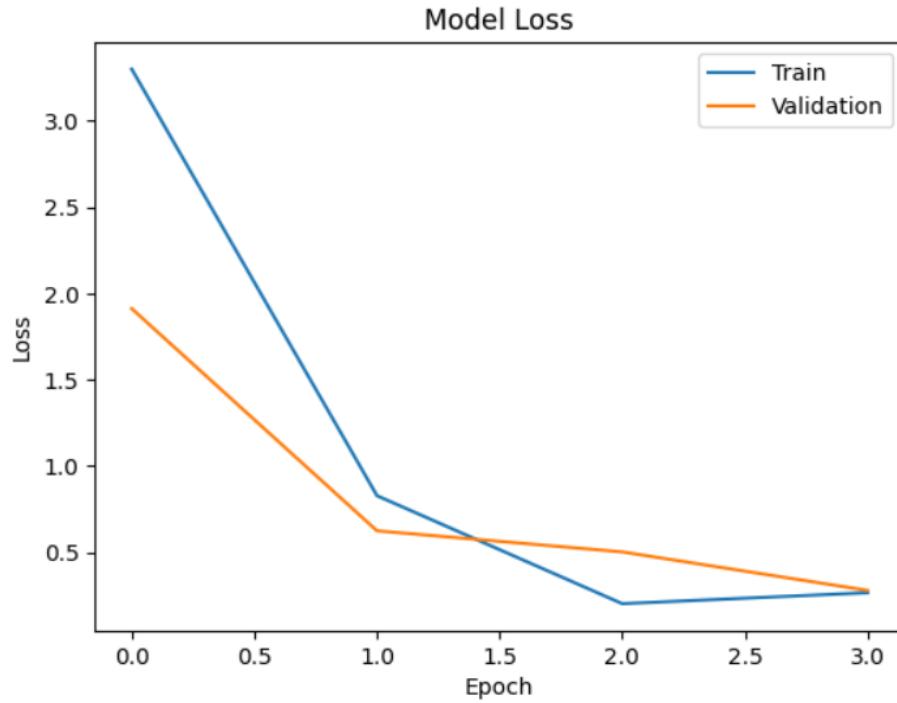


Fig 4.12 Loss plot for model5

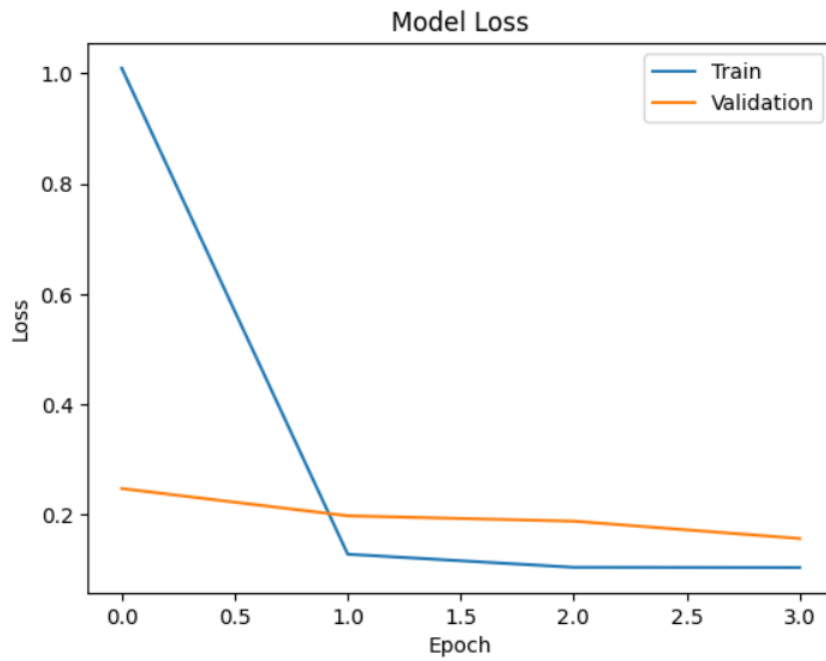


Fig 4.13 Loss plot for model6

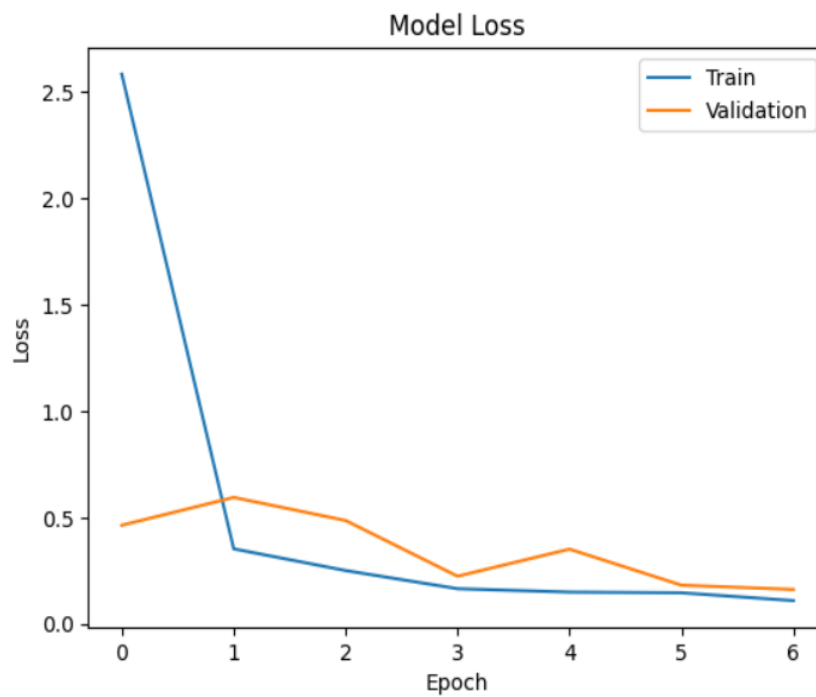


Fig 4.14 Loss plot for model7

4.3 MODEL EVALUATION:

Classification Report:				
	precision	recall	f1-score	support
0	0.50	0.53	0.51	100
1	0.50	0.47	0.48	100
accuracy			0.50	200
macro avg	0.50	0.50	0.50	200
weighted avg	0.50	0.50	0.50	200

Fig 4.15 Classification Report for model1

Classification Report:				
	precision	recall	f1-score	support
0	0.51	0.52	0.52	100
1	0.52	0.51	0.51	100
accuracy			0.52	200
macro avg	0.52	0.52	0.51	200
weighted avg	0.52	0.52	0.51	200

Fig 4.16 Classification Report for model2

Classification Report:				
	precision	recall	f1-score	support
0	0.50	0.52	0.51	100
1	0.51	0.49	0.50	100
accuracy			0.51	200
macro avg	0.51	0.51	0.50	200
weighted avg	0.51	0.51	0.50	200

Fig 4.17 Classification Report for model3

Classification Report:				
	precision	recall	f1-score	support
0	0.51	0.51	0.51	100
1	0.51	0.51	0.51	100
accuracy			0.51	200
macro avg	0.51	0.51	0.51	200
weighted avg	0.51	0.51	0.51	200

Fig 4.18 Classification Report for model4

Classification Report:				
	precision	recall	f1-score	support
0	0.53	0.58	0.56	100
1	0.54	0.49	0.51	100
accuracy			0.54	200
macro avg	0.54	0.53	0.53	200
weighted avg	0.54	0.54	0.53	200

Fig 4.19 Classification Report for model5

Classification Report:				
	precision	recall	f1-score	support
0	0.52	0.50	0.51	100
1	0.51	0.53	0.52	100
accuracy			0.52	200
macro avg	0.52	0.52	0.51	200
weighted avg	0.52	0.52	0.51	200

Fig 4.20 Classification Report for model6

Classification Report:					
		precision	recall	f1-score	support
	0	0.53	0.54	0.53	100
	1	0.53	0.52	0.53	100
	accuracy			0.53	200
	macro avg	0.53	0.53	0.53	200
	weighted avg	0.53	0.53	0.53	200

Fig 4.21 Classification Report for model7

4.4 CONFUSION MATRIX:

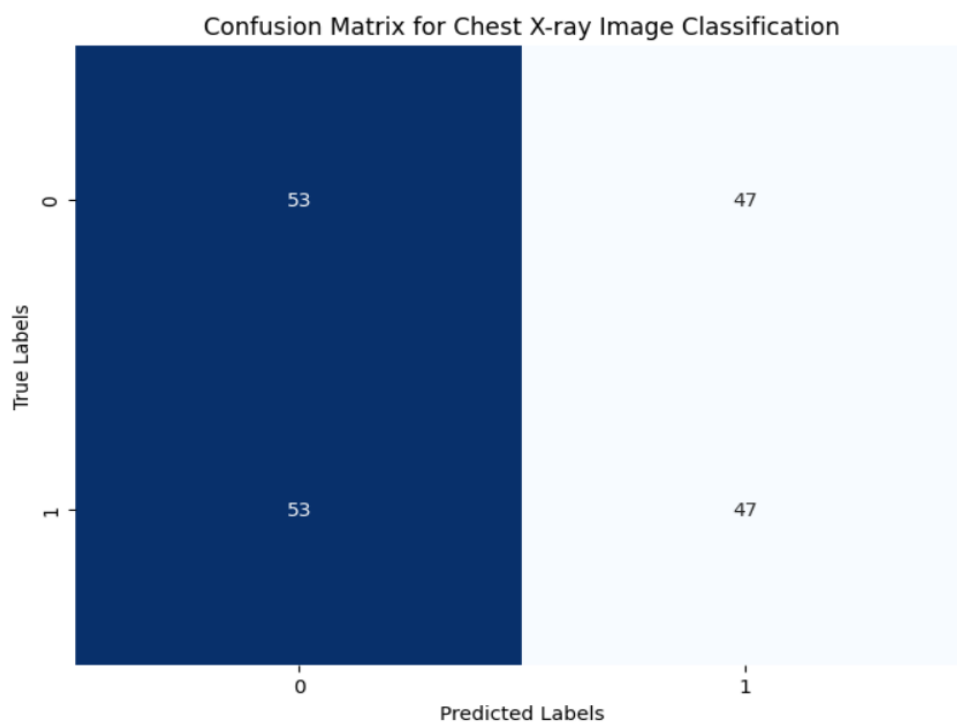


Fig 4.22 Confusion Matrix for model1

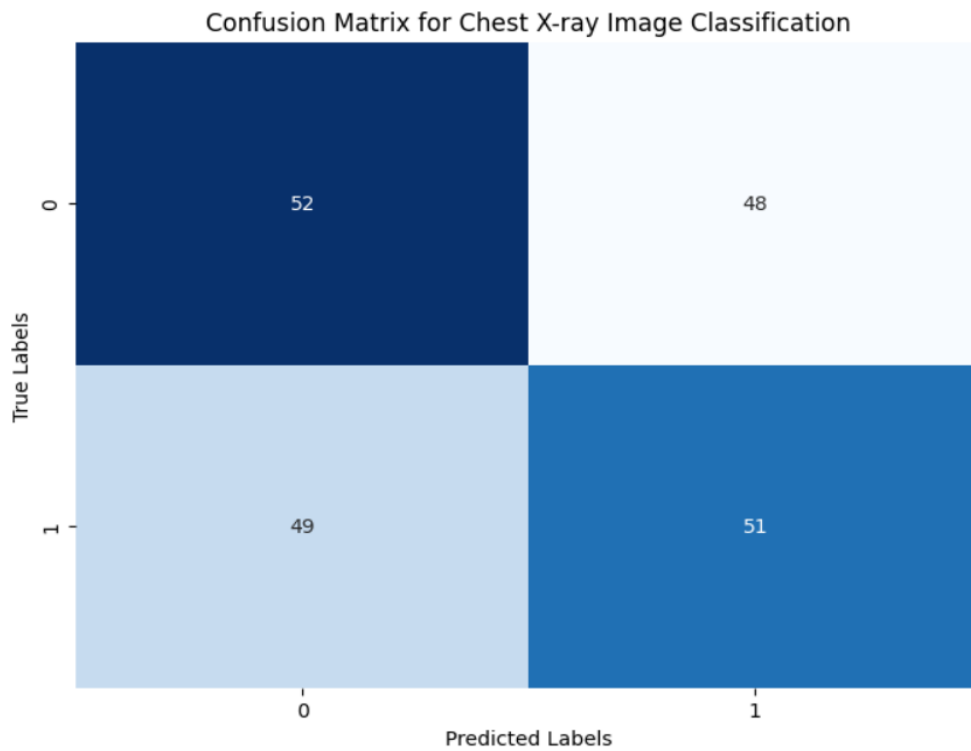


Fig 4.23 Confusion Matrix for model2

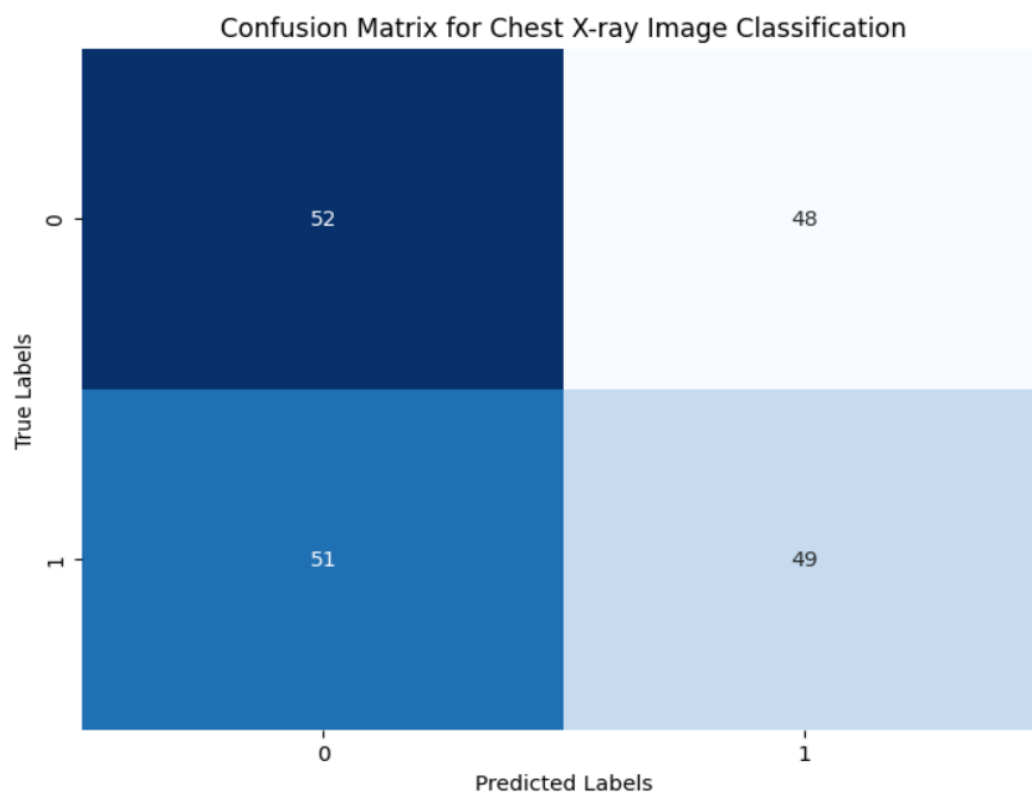


Fig 4.24 Confusion Matrix for model3

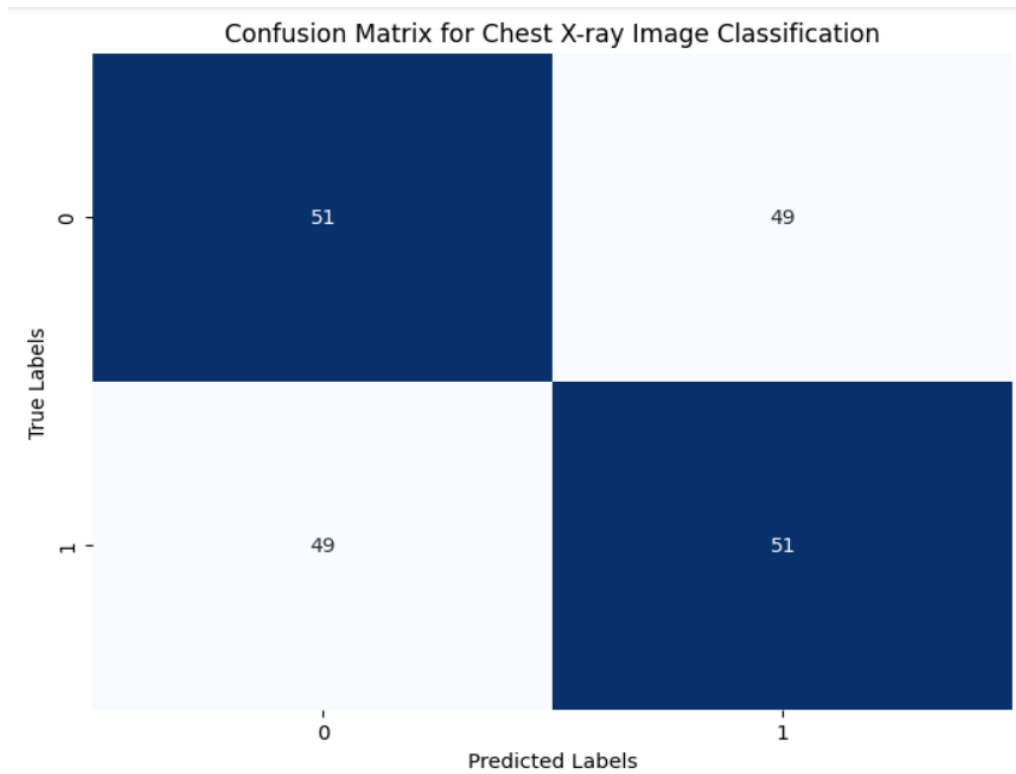


Fig 4.25 Confusion Matrix for model4

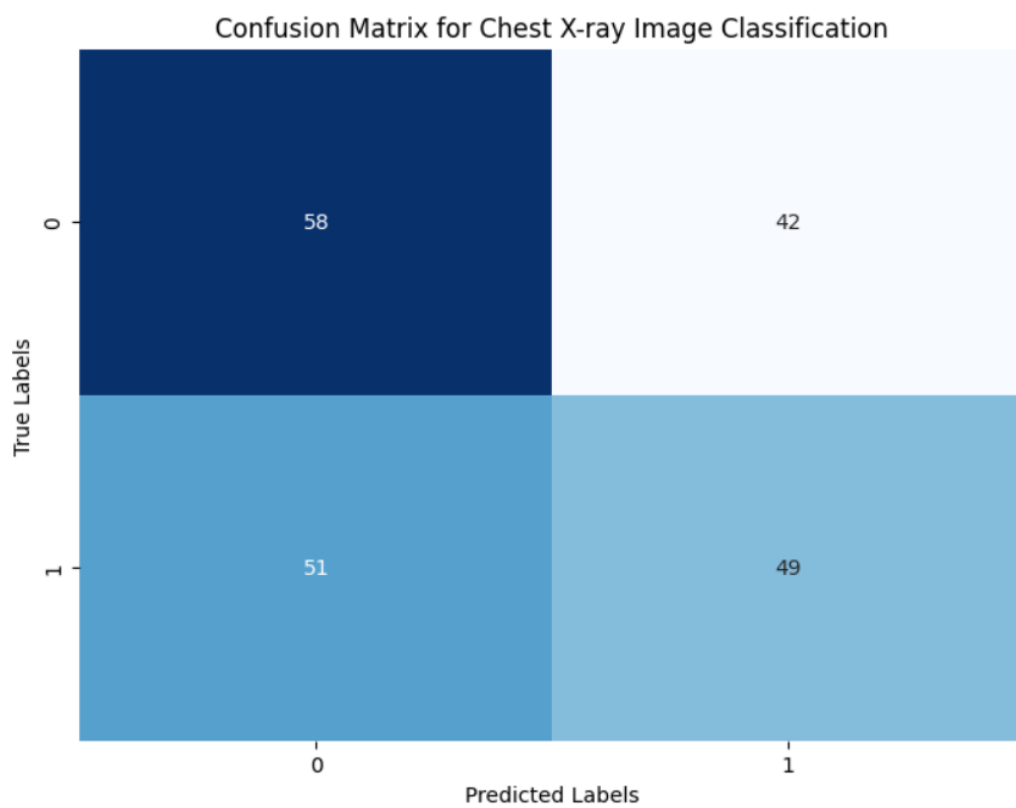


Fig 4.26 Confusion Matrix for model5

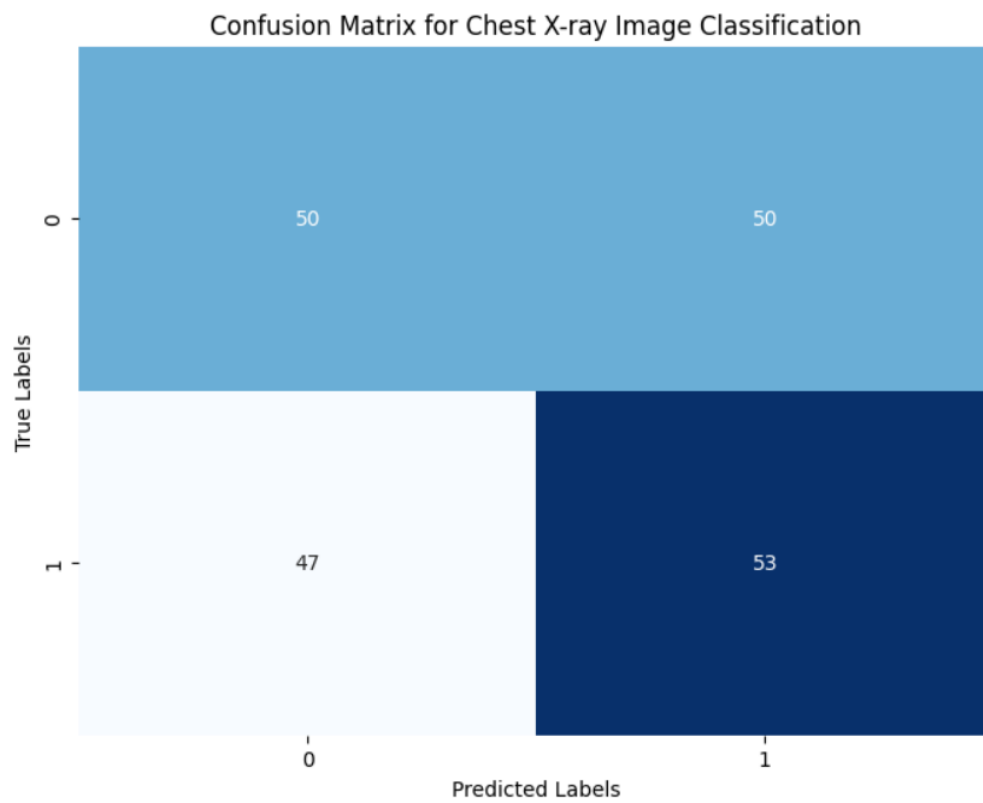


Fig 4.27 Confusion Matrix for model6

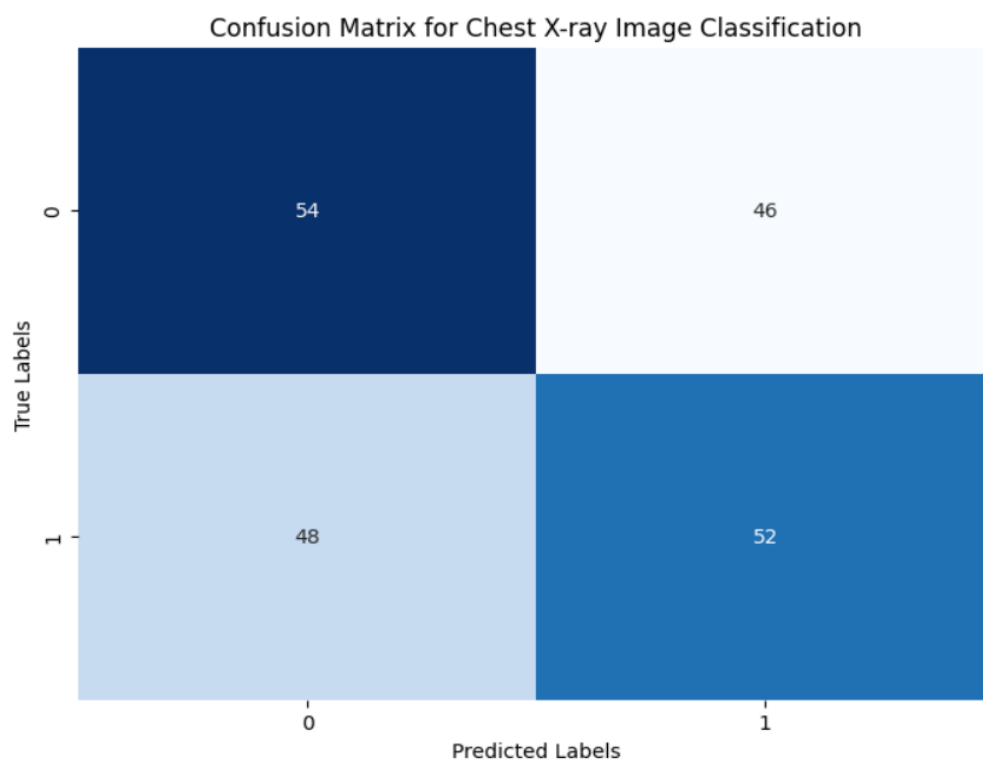


Fig 4.28 Confusion Matrix for model7

4.5 WEB APP:

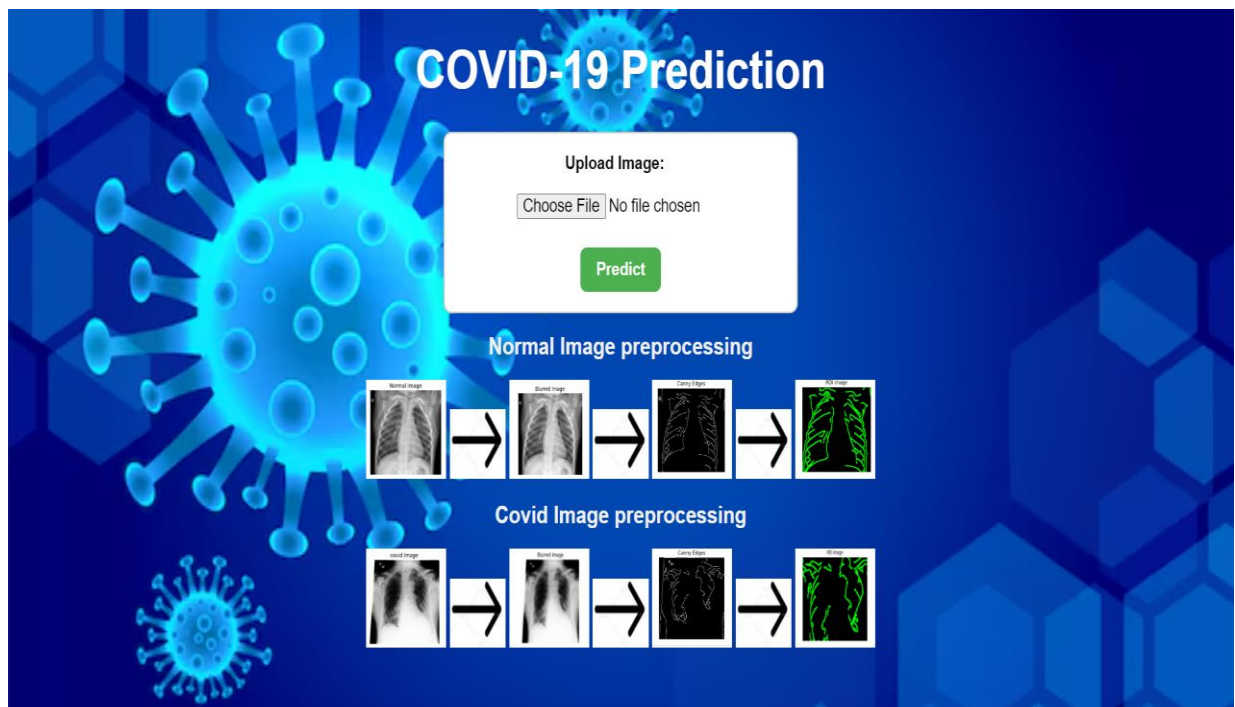


Fig 4.29 Image upload page



Fig 4.30 Data Attachment page

Prediction Results

Model	Class	Probability
RESNET50	Normal	(0.52)
DENSENET201	COVID-19	(1.00)
VGG-16	COVID-19	(0.94)
VGG-19	COVID-19	(0.99)
INCEPTIONV3	COVID-19	(0.61)
XCEPTION	COVID-19	(1.00)
MOBILENETV2	COVID-19	(0.98)
Best model	class	probability
XCEPTION	COVID-19	(1.00)

Fig 4.31 Result page for Covid-19 positive

Prediction Results

Model	Class	Probability
RESNET50	Normal	(0.53)
DENSENET201	Normal	(0.73)
VGG-16	Normal	(0.71)
VGG-19	COVID-19	(0.53)
INCEPTIONV3	COVID-19	(0.89)
XCEPTION	Normal	(1.00)
MOBILENETV2	COVID-19	(0.70)
Best model	class	probability
XCEPTION	Normal	(1.00)

Fig 4.32 Result page for Covid-19 negative

Table 4.1 presents a comprehensive overview of the calculated performance metrics for the models discussed above. These metrics provide valuable insights into the effectiveness and efficiency of the models in predicting and forecasting store item demand.

MODEL	MODEL NAME	ACCURACY	PRECESSION	RECALL	F1-SCORE
1	RESNET50	0.50	P:0.50 N:0.50	P:0.53 N:0.47	P:0.51 N:0.48
2	DENSENET201	0.52	P:0.51 N:0.52	P:0.52 N:0.51	P:0.52 N:0.51
3	VGG16	0.51	P:0.50 N:0.51	P:0.52 N:0.49	P:0.51 N:0.50
4	VGG19	0.51	P:0.51 N:0.51	P:0.51 N:0.51	P:0.51 N:0.51
5	XCEPTION	0.54	P:0.53 N:0.54	P:0.58 N:0.49	P:0.56 N:0.51
6	INCEPTION V3	0.52	P:0.52 N:0.51	P:0.50 N:0.53	P:0.51 N:0.52
7	MOBILENET V2	0.53	P:0.53 N:0.53	P:0.54 N:0.52	P:0.53 N:0.53

Table 4.1 Performance Analysis for various model

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 CONCLUSION:

This project culminated in the successful development of a web application for predicting COVID-19 cases. By leveraging the power of transfer learning, the application employed a pre-trained model, significantly expediting the development process while achieving superior predictive performance. The meticulously chosen transfer learning model, coupled with rigorously fine-tuned hyperparameters, demonstrably yielded effective results in predicting COVID-19 cases.

The user-centric design of the web application prioritizes ease of use, allowing users to effortlessly submit data and receive insightful predictions. This application has the potential to evolve into a valuable tool for a diverse user base, including researchers, healthcare professionals, and the general public. By providing forecasts on potential COVID-19 case trends, the application can empower informed decision-making across various sectors.

Overall, this project serves as a testament to the feasibility and effectiveness of utilizing transfer learning within a web application framework for COVID-19 prediction. As the global landscape surrounding the pandemic continues to evolve, this web application possesses the adaptability to be continuously refined and improved, ensuring its continued value as a valuable asset in the ongoing fight against COVID-19.

5.2 FUTURE WORK:

It involves assessing the generalizability of the model to real-world data. This necessitates developing the web application framework on a meticulously constructed dataset containing noise, outliers, and uncertainties that mirror real-world conditions. Evaluating the application's performance under such stresses will provide valuable insights into its ability to generalize and function robustly in practical settings. This rigorous testing phase will serve as the cornerstone for further refinements, ensuring the model's efficacy in handling the inherent complexities of real-world COVID-19 case prediction data.

To further enhance prediction accuracy, future endeavors should focus on optimizing the model's ability to capture temporal relationships within the data. This can be achieved through systematic experimentation with various lag step sizes. By meticulously adjusting these parameters, we can empower the model to identify and exploit a wider range of temporal patterns and dependencies present in the data. This optimization process has the potential to yield significant improvements in the accuracy of COVID-19 case predictions generated by the web application.

It involves exploring the integration of established time series models like ARIMA, SARIMA, ARIMAX, and SARIMAX with the current transfer learning framework. This hybrid approach would capitalize on the complementary strengths of each technique. Traditional time series models excel at capturing long-term trends and seasonal patterns, while transfer learning shines in extracting complex relationships from intricate data. By combining these methodologies, we have the potential to significantly improve the web application's overall forecasting capabilities, leading to more robust and accurate COVID-19 case predictions.

Future work can explore adapting the web application's core framework to other domains. This would require a deep dive into the specific data characteristics and demand patterns unique to each target industry, such as healthcare or transportation. By successfully adapting the framework to these diverse domains, the project's impact can be significantly amplified. The application has the potential to transform into a versatile tool, offering improved forecasting accuracy and empowering informed decision-making across a wider range of fields.

CHAPTER 6

REFERENCES

- [1] **Jain R, Gupta M, Taneja S , Hemanth DJ.** Deep learning based detection and analysis of covid-19 on chest x-ray. Appl Intell 2021.
- [2] **Das TK, Roy PK, Uddin M, Srinivasan K , Chang CY, Syed-Abdul S .** Early tumor diagnosis in brain mr images via deep convolutional neural network model. Comput Mater Countin 2021.
- [3] **Apostolopoulos ID, Mpesiana TA.** Covid-19: automatic detection from x-ray images utilizing transfer learning with convolutional neural networks. 2020, medRxiv Eng Sci Med 2020.
- [4] **Mohammed Rahimazadeh, Abolfazl Attar.** A modified deep convolutional neural network for detecting COVID-19 and pneumonia from chest X-ray images based on the concatenation of Xception and ResNet50V2, Informatics in Medicine Unlocked 2020.
- [5] **Loey M, Manogaran G, Khalifa NEM.** A deep transfer learning model with classical data augmentation and CGAN to detect COVID-19 from chest CT radiography digital images 2020.

CHAPTER 7

APPENDIX

7.1 SAMPLE CODE:

This chapter provides the source code for implementing different transfer learning models. Additionally, the code incorporates the use of various optimizers and allows for experimentation with different hyperparameters to optimize the model's performance. The various models implemented are given in Table 7.1.

Model no	Model	Optimizer	LR	Loss	Activation	Batch Size
1	RESNET50	Adam	10^{-4}	Binary-crossentropy	relu	32
2	DENSENET201	Adam	10^{-4}	Binary-crossentropy	relu	32
3	VGG16	Adam	10^{-4}	Binary-crossentropy	relu	32
4	VGG19	Adam	10^{-4}	Binary-crossentropy	relu	32
5	INCEPTIONV3	Adam	10^{-4}	Binary-crossentropy	relu	32
6	XCPETION	Adam	10^{-4}	Binary-crossentropy	relu	32
7	MOBILENETV2	Adam	10^{-4}	Binary-crossentropy	relu	32

Table 7.1 Parameter specifications for various models (LR- Learning Rate)

7.2.1 DATA LOADING, PREPARATION AND PREPROCESSING:

Import Libraries:

```
import os

import numpy as np

import cv2

import numpy as np

from tensorflow.keras.applications import ResNet50V2

from tensorflow.keras.layers import Dense, Flatten

from tensorflow.keras.models import Model

from tensorflow.keras.optimizers import Adam

from keras.callbacks import EarlyStopping

import numpy as np

from scipy import ndimage

import matplotlib.pyplot as plt

from keras.models import load_model

from sklearn.metrics import classification_report, confusion_matrix

from sklearn.metrics import roc_curve, auc

import joblib

from PIL import Image

from keras.preprocessing.image import ImageDataGenerator

from tensorflow.keras.preprocessing import image
```

Preprocessing:

```
def load_and_preprocess(folder_path):

    # Iterate through all the images in the folder
```

```

for filename in os.listdir(folder_path):

    if filename.endswith(".png"):

        # Load the X-ray image

        image = Image.open(os.path.join(folder_path, filename))

        # Noise Reduction: Gaussian Blur

        blurred_image = cv2.GaussianBlur(image, (5, 5), 0)

        # Edge Detection: Canny Edge Detector

        edges = cv2.Canny(blurred_image, 50, 150)

        # Find contours

        contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

        # Draw contours on a blank image

        contour_img = cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR)

        cv2.drawContours(contour_img, contours, -1, (0, 255, 0), 2)

        roi = contour_img[14:255 , 20:282]

        return roi

```

Train-Test Split:

```

from keras.preprocessing.image import ImageDataGenerator

train_path = "/content/drive/MyDrive/Train_withoutMask"

test_path = "/content/drive/MyDrive/Test_withoutMask"

batch_size = 32

# Data generators

train_datagen = ImageDataGenerator(

    rescale=1./255,

```

```

validation_split=0.2,

preprocessing_function=load_and_preprocess(train_path)
)

test_datagen =
ImageDataGenerator(rescale=1./255,preprocessing_function=load_and_preprocess(test_path))

# Train/validation generators

train_generator = train_datagen.flow_from_directory(

    train_path,

    target_size=(224, 224),

    batch_size=batch_size,

    class_mode='binary',

    subset='training'

)

validation_generator = train_datagen.flow_from_directory(

    train_path,

    target_size=(224, 224),

    batch_size=batch_size,

    class_mode='binary',

    subset='validation'

)

# Test generator

test_generator = test_datagen.flow_from_directory(

    test_path,

    target_size=(224, 224),

```

```

    batch_size=batch_size,

    class_mode='binary'

)

```

7.2 PROPOSED MODELS:

ResNet50:

```

# Load pre-trained Resnet model

base_model = ResNet50V2(weights='imagenet', include_top=False,
input_shape=(224, 224, 3))

# Freeze convolutional layers

for layer in base_model.layers:

    layer.trainable = False

# Add custom classification head

x = Flatten()(base_model.output)

x = Dense(256, activation='relu')(x)

output = Dense(1, activation='sigmoid')(x)

# Create model

model = Model(inputs=base_model.input, outputs=output)

# Define the optimizer with the specified learning rate

optimizer = Adam(learning_rate=learning_rate)

model.compile(optimizer=optimizer, loss='binary_crossentropy',metrics=['accuracy'])

es = EarlyStopping(monitor='val_loss', mode='max', verbose=1, patience=5)

H = model.fit(train_images_resized, train_labels,
validation_data=(val_images_resized, val_labels), epochs=10, verbose=1,
callbacks=[es])

```

DENSENET201:

```
# Load pre-trained Densenet201 model

base_model = DenseNet201(weights='imagenet', include_top=False,
input_shape=(224, 224, 3))

# Freeze convolutional layers

for layer in base_model.layers:

    layer.trainable = False

# Add custom classification head

x = Flatten()(base_model.output)

x = Dense(256, activation='relu')(x)

output = Dense(1, activation='sigmoid')(x)

# Create model

model = Model(inputs=base_model.input, outputs=output)

# Define the optimizer with the specified learning rate

optimizer = Adam(learning_rate=learning_rate)

model.compile(optimizer=optimizer, loss='binary_crossentropy', metrics=['accuracy'])

es = EarlyStopping(monitor='val_loss', mode='max', verbose=1, patience=5)

# Now, you can fit your model with the resized numpy arrays

H = model.fit(train_images_resized, train_labels,
validation_data=(val_images_resized, val_labels), epochs=10, verbose=1,
callbacks=[es])
```

VGG-16(with hyper parameter tuning):

```

vgg_model = VGG16(weights='imagenet', include_top=False, input_shape=(224,
224, 3))

for layer in vgg_model.layers:

    layer.trainable = False

def build_model(hp):

    model = Sequential([

        vgg_model,

        Flatten(),

        Dense(256, activation='relu'),

        Dense(1, activation='sigmoid') ])

    hp_learning_rate = hp.Choice('learning_rate', values=[1e-2, 1e-3, 1e-4])
    model.compile(optimizer=Adam(learning_rate=hp_learning_rate),

        loss='binary_crossentropy',

        metrics=['accuracy'])

    return model

tuner = RandomSearch(

    build_model,

    objective='val_accuracy',

    max_trials=2, # Adjust this as needed

    directory='tuner_results',

    project_name='vgg_tuning'

)

tuner.search(train_generator, validation_data=validation_generator,
epochs=5, batch_size=20)

# Get the best hyperparameters

```

```

best_hp = tuner.get_best_hyperparameters(num_trials=1)[0]

# Build the model with the best hyperparameters

best_model = tuner.hypermodel.build(best_hp)

es = EarlyStopping(monitor='val_loss', mode='max', verbose=1, patience=3)

# Train the best model

H = best_model.fit(train_generator,

                    validation_data=validation_generator,

                    epochs=10,

                    callbacks=[es])

```

VGG-19:

```

# Load pre-trained vgg19

base_model = VGG19(weights='imagenet', include_top=False, input_shape=(224,
224, 3))

# Freeze convolutional layers

for layer in base_model.layers:

    layer.trainable = False

# Add custom classification head

x = Flatten()(base_model.output)

x = Dense(256, activation='relu')(x)

output = Dense(1, activation='sigmoid')(x)

# Create model

model = Model(inputs=base_model.input, outputs=output)

# Define the optimizer with the specified learning rate

optimizer = Adam(learning_rate=learning_rate)

model.compile(optimizer=optimizer, loss='binary_crossentropy', metrics=['accuracy'])

```

```

es = EarlyStopping(monitor='val_loss', mode='max', verbose=1, patience=5)

# Now, you can fit your model with the resized numpy arrays

H = model.fit(train_images_resized, train_labels,
validation_data=(val_images_resized, val_labels), epochs=10, verbose=1,
callbacks=[es])

```

Inceptionv3(with hyperparameter tuning):

```

# Load pre-trained MobileNetV2 model

mobile_model = InceptionV3(weights='imagenet', include_top=False,
input_shape=(224, 224, 3))

# Freeze pre-trained layers

for layer in mobile_model.layers:

    layer.trainable = False

def build_model(hp):

    model = Sequential([

        mobile_model,

        Flatten(),

        Dense(units=hp.Int('units', min_value=256, max_value=1024, step=64),
activation='relu'),

        Dense(1, activation='sigmoid') ])

    hp_learning_rate = hp.Choice('learning_rate', values=[1e-2, 1e-3, 1e-4])

    model.compile(optimizer=Adam(learning_rate=hp_learning_rate),

        loss='binary_crossentropy',

        metrics=['accuracy'])

    return model

```

```

tuner = RandomSearch(

    build_model,

    objective='val_accuracy',

    max_trials=2, # Adjust this as needed

    directory='tuner2_results',

    project_name='InceptionV3_tuning')

tuner.search(train_generator, validation_data=validation_generator,
epochs=5,batch_size=20)

# Get the best hyperparameters

best_hp = tuner.get_best_hyperparameters(num_trials=1)[0]

# Build the model with the best hyperparameters

best_model = tuner.hypermodel.build(best_hp)

es = EarlyStopping(monitor='val_loss', mode='max', verbose=1, patience=3)

# Train the best model

H = best_model.fit(train_generator,

    validation_data=validation_generator,

    epochs=10,

    callbacks=[es] )

```

Xception(with hyperparameter tuning):

```

# Load pre-trained MobileNetV2 model

mobile_model = Xception(weights='imagenet', include_top=False, input_shape=(224,
224, 3))

# Freeze pre-trained layers

for layer in mobile_model.layers:

```

```

layer.trainable = False

def build_model(hp):

    model = Sequential([

        mobile_model,

        Flatten(),

        Dense(units=hp.Int('units', min_value=256, max_value=1024, step=64),
activation='relu'),

        Dense(1, activation='sigmoid')])

    hp_learning_rate = hp.Choice('learning_rate', values=[1e-2, 1e-3, 1e-4])

    model.compile(optimizer=Adam(learning_rate=hp_learning_rate),

        loss='binary_crossentropy',

        metrics=['accuracy'])

    return model

tuner = RandomSearch(

    build_model,

    objective='val_accuracy',

    max_trials=2, # Adjust this as needed

    directory='tuner_results',

    project_name='xception_tuning')

tuner.search(train_generator, validation_data=validation_generator,
epochs=5,batch_size=20)

# Get the best hyperparameters

best_hp = tuner.get_best_hyperparameters(num_trials=1)[0]

# Build the model with the best hyperparameters

best_model = tuner.hypermodel.build(best_hp)

```

```
es = EarlyStopping(monitor='val_loss', mode='max', verbose=1, patience=3)
```

```
# Train the best model
```

```
H = best_model.fit(train_generator,  
                   validation_data=validation_generator,  
                   epochs=10,  
                   callbacks=[es])
```

MobileNetv2:

```
base_model = MobileNetV2(weights='imagenet', include_top=False,  
input_shape=(224, 224, 3))
```

```
# Freeze convolutional layers
```

```
for layer in base_model.layers:
```

```
    layer.trainable = False
```

```
# Add custom classification head
```

```
x = Flatten()(base_model.output)
```

```
x = Dense(256, activation='relu')(x)
```

```
output = Dense(1, activation='sigmoid')(x)
```

```
# Create model
```

```
model = Model(inputs=base_model.input, outputs=output)
```

```
# Define the optimizer with the specified learning rate
```

```
optimizer = Adam(learning_rate=learning_rate)
```

```
model.compile(optimizer=optimizer, loss='binary_crossentropy', metrics=['accuracy'])
```

```
es = EarlyStopping(monitor='val_loss', mode='max', verbose=1, patience=5)
```

```
# Now, you can fit your model with the resized numpy arrays
```

```
H = model.fit(train_images_resized, train_labels,  
validation_data=(val_images_resized, val_labels), epochs=10, verbose=1,  
callbacks=[es])
```

Plot Between Accuracy and Epochs:

```
plt.plot(H.history['accuracy'])  
  
plt.plot(H.history['val_accuracy'])  
  
plt.title('Model accuracy')  
  
plt.xlabel('Epoch')  
  
plt.ylabel('accuracy')  
  
plt.legend(['Train', 'Validation'], loc='upper right')  
  
plt.show()
```

Plot Between Loss and Epochs:

```
# Plot training & validation loss  
  
plt.plot(H.history['loss'])  
  
plt.plot(H.history['val_loss'])  
  
plt.title('Model Loss')  
  
plt.xlabel('Epoch')  
  
plt.ylabel('Loss')  
  
plt.legend(['Train', 'Validation'], loc='upper right')  
  
plt.show()
```

Prediction:

```
# Define function to predict on an image

def predict_image(img_path):

    img = image.load_img(img_path, target_size=(224, 224)) # Adjust size if needed by
your model

    img = image.img_to_array(img)

    img = img / 255.0 # Normalize image (assuming model expects normalized input)

    img = np.expand_dims(img, axis=0) # Add batch dimension

    pred = best_model.predict(img)[0][0] # Get probability for positive class

    if pred > 0.5: # Adjust threshold as needed

        return "COVID-19 Positive ({:.2f}%)".format(pred * 100)

    else:

        return "COVID-19 Negative ({:.2f}%)".format((1 - pred) * 100)
```

7.3 PERFORMANCE MEASURES:**Classification Report:**

```
from sklearn.metrics import confusion_matrix, classification_report

import seaborn as sns

import matplotlib.pyplot as plt

y_true = test_generator.classes

y_pred = best_model.predict(test_generator)

y_pred_binary = (y_pred > 0.5).astype(int)

conf_matrix = confusion_matrix(y_true, y_pred_binary)

class_report = classification_report(y_true, y_pred_binary)

print("Classification Report:\n", class_report)
```

Confusion Matrix:

```
# Generate confusion matrix
```

```
print("Confusion Matrix:")  
  
print(confusion_matrix(y_true, y_pred_binary))
```

7.4 WEB APPLICATION:

HTML:

Page1.html

```
<!DOCTYPE html>  
  
<html lang="en">  
  
<head>  
  
    <meta charset="UTF-8">  
  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  
    <title>COVID-19 Prediction</title>  
  
</head>  
  
<body>  
  
    <header>  
  
        <h1>COVID-19 Prediction</h1>  
  
    </header>  
  
    <center>  
  
        <form id="prediction-form" method="POST" enctype="multipart/form-data"  
action="/">  
  
            <section class="prediction-form">  
  
                <label for="image">Upload Image:</label>  
  
                <input type="file" id="image" name="image"><br>  
  
                <button type="submit">Predict</button>  
  
            </section>  
  
        </form>
```

<section class ="preprocess">

<h2> Normal Image preprocessing</h2>

<img src ="C:/Users/DELL/OneDrive/Desktop/New folder
(5)/normal_image_input.png" width = 100px, height="100px">

<img src ="C:/Users/DELL/OneDrive/Desktop/New folder (5)/arrow.png"
width =70px , height="70px">

<img src ="C:/Users/DELL/OneDrive/Desktop/New folder
(5)/normal_image_blurred.png" width = 100px, height="100px">

<img src ="C:/Users/DELL/OneDrive/Desktop/New folder (5)/arrow.png"
width = 70px, height="70px">

<img src ="C:/Users/DELL/OneDrive/Desktop/New folder
(5)/normal_image_canny.png" width = 100px, height="100px">

<img src ="C:/Users/DELL/OneDrive/Desktop/New folder (5)/arrow.png"
width = 70px, height="70px">

<img src ="C:/Users/DELL/OneDrive/Desktop/New folder (5)/nor.png" width
= 100px, height="100px">

<h2> Covid Image preprocessing</h2>

<img src ="C:/Users/DELL/OneDrive/Desktop/New folder
(5)/covid_image_input.png" width = 100px, height="100px">

<img src ="C:/Users/DELL/OneDrive/Desktop/New folder (5)/arrow.png"
width =70px , height="70px">

<img src ="C:/Users/DELL/OneDrive/Desktop/New folder
(5)/covid_image_blur.png" width = 100px, height="100px">

<img src ="C:/Users/DELL/OneDrive/Desktop/New folder (5)/arrow.png"
width = 70px, height="70px">

<img src ="C:/Users/DELL/OneDrive/Desktop/New folder
(5)/covid_image_canny.png" width = 100px, height="100px">

```
<img src ="C:/Users/DELL/OneDrive/Desktop/New folder (5)/arrow.png"
width = 70px, height="70px">
```

```
<img src ="C:/Users/DELL/OneDrive/Desktop/New folder
(5)/covid_image_roi.png" width = 100px, height="100px">
```

```
</section>
```

```
</center>
```

```
</body>
```

```
</html>
```

Second_page.html:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Prediction Results</title>
```

```
</head>
```

```
<body>
```

```
<header>
```

```
<h1>Prediction Results</h1>
```

```
</header>
```

```
<section class="prediction-results" id="prediction-results">
```

```
<table>
```

```
<thead>
```

```
<tr>
```

```
<th>Model</th>
```

```

        <th>Class</th>

        <th>Probability</th>

        <th> accuracy</th>

    </tr>

</thead>

<tbody>

    <tr>

        <td>RESNET50</td>

        <td>{{ predicted_class["model1"].split(" ")[0] }}</td>

        <td>{{ predicted_class["model1"].split(" ")[-1] }}</td>

        <td> {{ acc1 }}</td>

    </tr>

    <tr>

        <td>DENSENET201</td>

        <td>{{ predicted_class["model2"].split(" ")[0] }}</td>

        <td>{{ predicted_class["model2"].split(" ")[-1] }}</td>

    </tr>

    <tr>

        <td>VGG-16</td>

        <td>{{ predicted_class["model3"].split(" ")[0] }}</td>

        <td>{{ predicted_class["model3"].split(" ")[-1] }}</td>

    </tr>

    <tr>

```

```

        <td>VGG-19</td>

        <td>{{ predicted_class["model4"].split(" ")[0] }}</td>

        <td>{{ predicted_class["model4"].split(" ")[-1] }}</td>

    </tr>

    <tr>

        <td>INCEPTIONV3</td>

        <td>{{ predicted_class["model5"].split(" ")[0] }}</td>

        <td>{{ predicted_class["model5"].split(" ")[-1] }}</td>

    </tr>

    <tr>

        <td>XCEPTION</td>

        <td>{{ predicted_class["model6"].split(" ")[0] }}</td>

        <td>{{ predicted_class["model6"].split(" ")[-1] }}</td>

    </tr>

    <tr>

        <td>MOBILENETV2</td>

        <td>{{ predicted_class["model7"].split(" ")[0] }}</td>

        <td>{{ predicted_class["model7"].split(" ")[-1] }}</td>

    </tr>

</tbody>

</table>

</section>

<center>

```

```

<table>

  <tr>

    <th> Best model</th>

    <th> class</th>

    <th> probability </th>

  </tr>

  <tr>

    <td>XCEPTION</td>

    <td>{{ predicted_class["model6"].split(" ")[0] }}</td>

    <td>{{ predicted_class["model6"].split(" ")[-1] }}</td>

  </tr>

</table>

</center>

</body>

</html>

```

Error.html

```

<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Prediction Results</title>

</head>

```

```
<body>
```

ERRORR FILE NOT SUPPORTED

```
</body>
```

```
</html>
```

CSS:

Page1.css:

```
h1 {  
    font-size: 50px;  
    font-style: normal;  
    font-weight: 500;  
    background-image: linear-gradient(to bottom left, #793c9a, #ee2bba);  
    color: transparent;  
    background-clip: text;  
    text-align: center;  
    color:white;  
    animation: animateText 1.5s ease-in-out forwards;  
    font-family: 'Roboto Condensed',sans-serif;  
    font-weight: bold;  
}  
  
h2{  
    color:#f0f0f0;  
}  
  
@keyframes animateText  
{
```

```

from{

    opacity: 0;

    transform: scale(1);

}

to{

    opacity:1;

    transform: scale(1.05);

}

}

body {

    font-family: Arial, sans-serif;

    margin: 0;

    padding: 0;

    background-color: #f0f0f0; /* Light gray background */

    background-image: url("https://img.freepik.com/free-vector/blue-coronavirus-covid-19-pandemic-outbreak-background-design_1017-24425.jpg?size=626&ext=jpg");

    background-size:cover;

    background-position: center;

}

#prediction-form {

    margin-top: 20px;

    padding: 20px;

    border: 2px solid #ccc;

    border-radius: 10px;

```

```
width: 400px;

background-color: #fff; /* White background for the form */

transform: translateX(0px);

transition: transform 0.2s ease-in-out;

box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);

}
```

```
.prediction-form p {

    margin-bottom: 15px;

    font-style: italic;

}
```

```
.prediction-form label {

    display: block;

    margin-bottom: 10px;

    padding-right: 15px;

    font-size: large;

    font-family: 'Roboto Condensed', sans-serif;

    font-weight: bold;

}
```

```
.prediction-form input[type="file"] {

    margin-bottom: 10px;

    padding: 10px;

    padding-left: 70px;

    font-size: large;
```

```

    align-items: center;
}

.prediction-form button {
    background-color: #4CAF50;
    color: white;
    padding: 12px 20px;
    border: none;
    border-radius: 8px;
    cursor: pointer;
    font-size: large;
    transition: background-color 0.2s ease-in-out;
    font-family: 'Roboto Condensed', sans-serif;
    font-weight: bold;
}

.prediction-form button:hover {
    background-color: #45a049;
    animation-play-state: paused;
    transform: translateX(5px);
    box-shadow: 0 0 5px #45a049, 0 0 25px #45a049, 0 0 50px #000000, 0 0
100px #000000 inset;
}

@media(max-width: 768px)
{
    h1{
        font-size: 36px;
    }
}

```

```

    }

    #prediction-form{

        width:90%;

    }

    .prediction-form label,

    .prediction-form input[type='file'],

    .prediction-form button{

        font-size: medium;

    }

}

@media (max-width:480px){

    h1{

        font-size: 28px;

    }

    .prediction-form label

    {

        padding-right: 5px;

    }

}

#image {

    display: block;

    margin: 0 auto;

    text-align: center;

}

```

Secondpage.css:

```
body {  
  
    font-family: Arial, sans-serif; /* Set a clean, modern font */  
  
    margin: 0;  
  
    padding: 0;  
  
    background-color: #f5f5f5; /* Light background */  
  
}  
  
header {  
  
    text-align: center;  
  
    padding: 20px 0;  
  
    background-color: #fff; /* White background for header */  
  
    box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1); /* Subtle shadow for depth */  
  
}  
  
h1 {  
  
    font-size: 50px;  
  
    font-style: normal;  
  
    font-weight: 500;  
  
    background-image: linear-gradient(to bottom left, magenta, #ee2bba);  
  
    color: transparent;  
  
    background-clip: text;  
  
    text-align: center;  
  
}  
  
section.prediction-results {  
  
    display: flex; /* Center content horizontally */
```

```

    justify-content: center;

    align-items: center;

    min-height: 300px; /* Set a minimum height to avoid content squeezing */
}

#prediction-text {

    color: #333; /* Darker text color for readability */

    font-size: 24px;

    text-align: center;

    line-height: 1.5; /* Increase line spacing for better visual hierarchy */
}

#prediction-text {

    animation: text-reveal 2s ease-in-out forwards;
}

@keyframes text-reveal {

    from { opacity: 0; }

    to { opacity: 1; }

}

```

Flask:

```

import pickle

import bz2

import tensorflow

from flask import Flask,render_template,request

import cv2,os

import numpy as np

```

```

import tensorflow as tf

import time

app = Flask(__name__,template_folder='template' )

def load_h5_model(filepath):

    """Loads a Keras model from an HDF5 file."""

    with h5py.File(filepath, 'r') as f:

        model = tf.keras.models.load_model(f)

    return model

# Load the model using load_h5_model

model1 = load_h5_model(r"ROI_Resnet50.h5")

model2 = load_h5_model(r"ROI_Densenet201.h5")

model3 = load_h5_model(r"vgg16_roi.h5")

model4 = load_h5_model(r"ROI_Vgg19.h5")

model5 = load_h5_model(r"Inceptionv3_roi.h5")

model6 = load_h5_model(r"Xception_roi.h5")

model7 = load_h5_model(r"ROI_mobilenetV2.h5")

# Function to classify a single image

def classify_single_image(image_path):

    # Read the image

    img = cv2.imread(image_path)

    # Resize the image to match the desired input shape

    resized_img = cv2.resize(img, (224, 224))

    # Preprocess the image

    processed_img = resized_img.astype('float32') / 255.0

```

```

processed_img = np.expand_dims(processed_img, axis=0) # Add batch dimension

# Predict probabilities

pred1 = model1.predict(processed_img)
pred2 = model2.predict(processed_img)
pred3 = model3.predict(processed_img)
pred4 = model4.predict(processed_img)
pred5 = model5.predict(processed_img)
pred6 = model6.predict(processed_img)
pred7 = model7.predict(processed_img)

# Extract the first element (assuming single value)

predicted_value1 = pred1.item()
predicted_value2 = pred2.item()
predicted_value3 = pred3.item()
predicted_value4 = pred4.item()
predicted_value5 = pred5.item()
predicted_value6 = pred6.item()
predicted_value7 = pred7.item()

# Set a threshold for classification

threshold = 0.5

predictions = {

    "model1": f"COVID-19 ({predicted_value1:.2f})" if predicted_value1 > threshold
else f"Normal ({(1 - predicted_value1):.2f})",

    "model2": f"COVID-19 ({predicted_value2:.2f})" if predicted_value2 > threshold
else f"Normal ({(1 - predicted_value2):.2f})",

```

```
"model3": f"COVID-19 ({predicted_value3:.2f})" if predicted_value3 > threshold
else f"Normal ({(1 - predicted_value3):.2f})",
```

```
"model4": f"COVID-19 ({predicted_value4:.2f})" if predicted_value4 > threshold
else f"Normal ({(1 - predicted_value4):.2f})",
```

```
"model5": f"COVID-19 ({predicted_value5:.2f})" if predicted_value5 > threshold
else f"Normal ({(1 - predicted_value5):.2f})",
```

```
"model6": f"COVID-19 ({predicted_value6:.2f})" if predicted_value6 > threshold
else f"Normal ({(1 - predicted_value6):.2f})",
```

```
"model7": f"COVID-19 ({predicted_value7:.2f})" if predicted_value7 > threshold
else f"Normal ({(1 - predicted_value7):.2f})"
```

```
}0
```

```
@app.route('/', methods=['GET', 'POST'])
```

```
def upload_file():
```

```
    if request.method == 'POST':
```

```
        # Get the uploaded image file
```

```
        image = request.files['image']
```

```
        # Check for valid image extension
```

```
        allowed_extensions = {'jpg', 'jpeg', 'png'} # Adjust as needed
```

```
        if image.filename.split('.')[1].lower() not in allowed_extensions:
```

```
            return render_template('error.html', message="Invalid image format")
```

```
        # Save the file locally with a unique filename
```

```
        filename = f"{int(time.time())}.{image.filename.split('.')[1]}" # Use timestamp
and extension
```

```
        # Use the image path for model prediction
```

```
        predicted_class = classify_single_image(image_path)
```

```
        return render_template('second_page.html', predicted_class=predicted_class)
```

```
    return render_template('page1.html')

if __name__ == '__main__':

    app.run(debug=True)
```